

CS 2569 / C4-2

เอกสารคู่มือการใช้งาน

ไดโน : เว็บแอพลิเคชันศูนย์กลางของการท่องเที่ยวจังหวัดขอนแก่น

Dino: The central web application for tourism in Khon Kaen

โดย

663380203-9 นายคเณศ บัวสด
663380228-3 นายภูริ ชาริชา

อาจารย์ที่ปรึกษา: อ.ธนพล ตั้งชูพงศ์

รายงานนี้เป็นส่วนหนึ่งของการศึกษาวิชา CP354 771 โครงงานทางวิทยาการคอมพิวเตอร์ 1
ภาคเรียนที่ 1 ปีการศึกษา 2569
สาขาวิชาวิทยาการคอมพิวเตอร์ วิทยาลัยการคอมพิวเตอร์
มหาวิทยาลัยขอนแก่น
(เดือน ตุลาคม พ.ศ. 2569)

คำนำ

เอกสารคู่มือการใช้งานเว็บแอปพลิเคชัน “ไดโน” ได้จัดทำขึ้นโดยมีวัตถุประสงค์เพื่อให้ผู้ใช้งานสามารถใช้งานระบบได้อย่างถูกต้องและเกิดประสิทธิภาพสูงสุด คู่มือนี้ประกอบด้วยขั้นตอนการเข้าใช้งาน การสมัครสมาชิกและเข้าสู่ระบบ การค้นหาสถานที่ท่องเที่ยวและกิจกรรม การสอบถามข้อมูลกับแชตบอต การสร้างและปรับแผนการเดินทาง การสะสมพอยท์ด้วยการสแกน QR Code ตลอดจนการใช้งานในส่วนของผู้ดูแลระบบ และขั้นตอนการติดตั้งระบบสำหรับนักพัฒนา ผู้จัดทำหวังเป็นอย่างยิ่งว่าคู่มือฉบับนี้จะเป็นประโยชน์ต่อผู้ใช้งานทุกกลุ่ม

ผู้จัดทำ

นาย คเณศ บัวสด

นาย ภูริ ชาริชา

สารบัญ

คำนำ ก

สารบัญ ข

สารบัญภาพ ง

สารบัญตาราง ฉ

1 บทนำ **1**

1.1 ความเป็นมาและความสำคัญของปัญหา 1

1.2 เครื่องมือที่ใช้พัฒนา 2

1.2.1 React และ Vite 2

1.2.2 Node.js และ Express 2

1.2.3 Python และ FastAPI 2

1.2.4 PostgreSQL บน Supabase 2

1.2.5 บริการภายนอก 2

1.2.6 Visual Studio Code 2

1.3 ขอบเขตและข้อจำกัด 2

1.3.1 ขอบเขตของผู้ใช้งานทั่วไป (Guest) 2

1.3.2 ขอบเขตของสมาชิก (Tourist) 3

1.3.3 ขอบเขตของผู้ดูแลระบบ (Admin) 3

1.3.4 ข้อจำกัดของระบบ 3

2 คู่มือการใช้งาน **4**

2.1 คู่มือการใช้งาน 4

2.1.1 เข้าใช้งานกรณีเป็นผู้ใช้งานทั่วไป 4

2.1.2 เข้าใช้งานกรณีเข้าสู่ระบบของสมาชิก 12

2.1.3 เข้าใช้งานกรณีเข้าสู่ระบบของผู้ดูแลระบบ 17

3 การติดตั้งระบบสำหรับพัฒนา **25**

3.1 ความต้องการของระบบ 25

3.1.1 ระบบปฏิบัติการ 25

3.1.2 ซอฟต์แวร์และบริการที่ต้องเตรียม 25

3.2 ขั้นตอนการติดตั้งระบบสำหรับนักพัฒนา 25

3.2.1 การติดตั้ง Node.js 25

3.2.2 การติดตั้ง Python 26

3.2.3 การติดตั้ง Visual Studio Code และโครงสร้างโปรเจกต์ . . 27

3.2.4	การดาวน์โหลดซอร์สโค้ดจาก GitHub	27
3.2.5	การสร้างฐานข้อมูลบน Supabase	27
3.2.6	การติดตั้งไลบรารีของแต่ละส่วน	28
3.2.7	การตั้งค่าไฟล์ .env	28
3.2.8	การเชื่อมต่อโปรเจกต์ Supabase ด้วย Supabase CLI . . .	31
3.2.9	การนำเข้าข้อมูลตั้งต้น	31
3.2.10	การสร้างบัญชีผู้ดูแลระบบ	32
3.2.11	การเริ่มการทำงานของระบบ	32
3.2.12	ภาพรวมการติดตั้งระบบเพื่อใช้งานจริง	32
3.2.13	การเตรียมเซิร์ฟเวอร์และไฟล์ตั้งค่า	33
3.2.14	การสร้างและเริ่มระบบด้วย Docker Compose	34
3.2.15	การปรับปรุงระบบด้วยสคริปต์ deploy.sh	35
3.2.16	การตรวจสอบและนำระบบขึ้นอัตโนมัติด้วย GitHub Actions	35

สารบัญภาพ

ภาพที่ 1 หน้าแรกและแถบเมนูผู้ใช้งานทั่วไป 4
ภาพที่ 2 หน้าฟอร์มสมัครสมาชิก 5
ภาพที่ 3 หน้าฟอร์มเข้าสู่ระบบ 6
ภาพที่ 4 หน้าลืมรหัสผ่าน 6
ภาพที่ 5 หน้ารายการสถานที่ท่องเที่ยว 7
ภาพที่ 6 หน้ารายละเอียดสถานที่ท่องเที่ยว 7
ภาพที่ 7 หน้ารายการกิจกรรมและเทศกาล 8
ภาพที่ 8 หน้ารายละเอียดกิจกรรม 8
ภาพที่ 9 หน้าต่างแชตบอตน้องไดโน 9
ภาพที่ 10 หน้าแบบฟอร์มสร้างแผนการเดินทาง ขั้นตอนที่ 1 วันเดินทาง 9
ภาพที่ 11 หน้าแบบฟอร์มสร้างแผนการเดินทาง ขั้นตอนที่ 2 ความสนใจ 10
ภาพที่ 12 หน้าแบบฟอร์มสร้างแผนการเดินทาง ขั้นตอนที่ 3 งบประมาณ 10
ภาพที่ 13 หน้าแบบฟอร์มสร้างแผนการเดินทาง ขั้นตอนที่ 4 ช่วงเวลา 11
ภาพที่ 14 หน้าสถานะกำลังประมวลผลแผนการเดินทาง 11
ภาพที่ 15 หน้าแสดงผลแผนการเดินทาง 11
ภาพที่ 16 หน้าพอยท์สะสม กรณีผู้ใช้งานทั่วไป 12
ภาพที่ 17 แถบเมนูสมาชิก 13
ภาพที่ 18 หน้าพอยท์สะสมของสมาชิก 13
ภาพที่ 19 หน้าต่างสแกน QR Code 14
ภาพที่ 20 หน้าทริปของฉัน 14
ภาพที่ 21 หน้าโปรไฟล์ แท็บภาพรวม 15
ภาพที่ 22 หน้าโปรไฟล์ แท็บข้อมูล 15
ภาพที่ 23 หน้าโปรไฟล์ แท็บประวัติ 16
ภาพที่ 24 หน้าโปรไฟล์ แท็บกิจกรรมของฉัน 16
ภาพที่ 25 หน้าโปรไฟล์ แท็บความปลอดภัย 16
ภาพที่ 26 หน้าเข้าสู่ระบบสำหรับผู้ดูแลระบบ 17
ภาพที่ 27 หน้าแดชบอร์ดผู้ดูแลระบบ 18
ภาพที่ 28 หน้าจัดการสถานที่ 18
ภาพที่ 29 หน้าฟอร์มเพิ่มและแก้ไขข้อมูลสถานที่ 19
ภาพที่ 30 หน้าจัดการกิจกรรม 19
ภาพที่ 31 หน้าแบบฟอร์มกิจกรรมพร้อมเครื่องมือสกัดข้อมูลจากข้อความโพสต์ด้วย AI 20
ภาพที่ 32 หน้าจัดการฐานความรู้แชตบอต 20
ภาพที่ 33 หน้ารายงานข้อมูล 21
ภาพที่ 34 หน้าคำขอกิจกรรม 21
ภาพที่ 35 หน้าจัดการ QR Code 22
ภาพที่ 36 หน้าจัดการของรางวัล 22
ภาพที่ 37 หน้าแลกของรางวัลที่เคาน์เตอร์ 23
ภาพที่ 38 หน้าจัดการผู้ใช้ 23

สารบัญภาพ (ต่อ)

ภาพที่ 39 หน้าดูทริปของผู้ใช้ 24
ภาพที่ 40 หน้าผลความพึงพอใจ 24
ภาพที่ 41 หน้าดาวน์โหลด Node.js 26
ภาพที่ 42 หน้าดาวน์โหลด Python 26

สารบัญตาราง

ตารางที่ 1 โครงสร้างโฟลเดอร์ของโครงงาน 27
ตารางที่ 2 ตัวแปรสภาพแวดล้อมที่ใช้ในไฟล์ .env 29
ตารางที่ 3 บริการที่ Docker Compose จัดการ 33
ตารางที่ 4 ไฟล์ตั้งค่าสำหรับการใช้งานจริงในโฟลเดอร์ deploy 34
ตารางที่ 5 ขั้นตอนอัตโนมัติของ GitHub Actions 36
ตารางที่ 6 ค่าลับของ GitHub Actions สำหรับการนำระบบขึ้นอัตโนมัติ 36

บทที่ 1
บทนำ

1.1 ความเป็นมาและความสำคัญของปัญหา

จังหวัดขอนแก่นนับเป็นจังหวัดที่มีศักยภาพทางการท่องเที่ยวสูง ด้วยความหลากหลายของแหล่งท่องเที่ยว ทั้งทางธรรมชาติ วัฒนธรรม ประวัติศาสตร์ ตลอดจนแหล่งท่องเที่ยวเชิงวิชาการที่โดดเด่นอย่าง “ไดโนเสาร์” ซึ่งถือเป็นอัตลักษณ์ที่สร้างชื่อเสียงให้กับจังหวัด ขอนแก่นจึงเป็นหนึ่งในจุดหมายสำคัญของนักท่องเที่ยวทั้งชาวไทยและต่างชาติที่ต้องการสัมผัสประสบการณ์ที่หลากหลายและแตกต่าง

อย่างไรก็ตาม ปัญหาสำคัญที่ยังคงปรากฏอยู่คือ การเข้าถึงข้อมูลการท่องเที่ยวที่กระจัดกระจายอยู่ในหลายช่องทาง ไม่ว่าจะเป็นเว็บไซต์ หน่วยงานราชการ เพจโซเชียลมีเดีย หรือแอปพลิเคชันทั่วไป นักท่องเที่ยวจำเป็นต้องใช้เวลาค้นหาข้อมูลจากหลายแหล่ง ซึ่งมักพบปัญหาว่าข้อมูลไม่ครบถ้วน ไม่ทันสมัย หรือขาดการรองรับภาษาสำหรับนักท่องเที่ยวต่างชาติ ส่งผลให้การวางแผนท่องเที่ยวไม่มีประสิทธิภาพและอาจทำให้นักท่องเที่ยวพลาดโอกาสในการเข้าถึงประสบการณ์ที่ควรได้รับ

เพื่อแก้ไขข้อจำกัดดังกล่าว จึงเกิดแนวคิดในการพัฒนาโครงงาน “ไดโน: เว็บแอปพลิเคชันศูนย์กลางของการท่องเที่ยวจังหวัดขอนแก่น” ซึ่งเป็นแอปพลิเคชันที่มุ่งเน้นการรวมศูนย์ข้อมูลการท่องเที่ยวอย่างครบวงจร ภายในแอปประกอบด้วยฟังก์ชันการค้นหาแหล่งท่องเที่ยว ร้านอาหาร ที่พัก และกิจกรรมต่าง ๆ ระบบวางแผนการเดินทางที่จัดลำดับสถานที่ให้สอดคล้องกับระยะเวลา ความสนใจ และงบประมาณของผู้ใช้ รวมถึงระบบ แชตบอต ที่ตอบคำถามโดยอ้างอิงเฉพาะข้อมูลในฐานความรู้ของระบบ ช่วยให้การค้นหาข้อมูลและการวางแผนทริปเป็นไปอย่างสะดวก รวดเร็ว และทันสมัย

นอกจากการอำนวยความสะดวกแก่นักท่องเที่ยวแล้ว แอปพลิเคชันยังเป็นเครื่องมือสำคัญในการสนับสนุนผู้ประกอบการท้องถิ่นให้สามารถโปรโมทธุรกิจได้ตรงกลุ่มเป้าหมาย อีกทั้งยังสามารถเก็บสถิติการใช้งาน เช่น จุดท่องเที่ยวยอดนิยม ช่วงฤดูกาลท่องเที่ยว หรือกิจกรรมที่ได้รับความสนใจสูง ซึ่งข้อมูลเหล่านี้สามารถนำไปใช้วิเคราะห์แนวโน้มและวางแผนการพัฒนาการท่องเที่ยวของจังหวัดในอนาคต

ดังนั้น การพัฒนาแอปพลิเคชัน “ไดโน” จึงไม่เพียงเป็นการแก้ปัญหาด้านการเข้าถึงข้อมูลการท่องเที่ยวเท่านั้น แต่ยังมีบทบาทสำคัญในการสร้างประสบการณ์การท่องเที่ยวที่ครบถ้วนสำหรับผู้มาเยือน และสนับสนุนเศรษฐกิจท้องถิ่นอย่างยั่งยืน ทั้งยังเป็นการวางรากฐานที่มั่นคงต่อการพัฒนาการท่องเที่ยวของจังหวัดขอนแก่นในระยะยาว

1.2 เครื่องมือที่ใช้พัฒนา

เว็บแอปพลิเคชัน “ไดโน” (Dino Khon Kaen) เผยแพร่ใช้งานจริงที่ https://dinokhonkaen.app พัฒนาเป็นบริการอิสระ 3 ส่วน ได้แก่ ส่วนติดต่อผู้ใช้งาน บริการข้อมูลหลัก และ AI Service ซึ่งแยกโฟลเดอร์และทำงานแยกจากกัน โดยใช้เครื่องมือดังนี้

1.2.1 React และ Vite

ใช้พัฒนาส่วนติดต่อผู้ใช้งาน (โฟลเดอร์ `frontend`) ซึ่งเป็นเว็บแอปพลิเคชันชุดเดียวที่ให้บริการทั้งฝั่งผู้ใช้งานและหน้าผู้ดูแลระบบ ใช้ React 18 ร่วมกับ react-router-dom และใช้ Vite เป็นเครื่องมือพัฒนาและสร้างไฟล์สำหรับใช้งานจริง

1.2.2 Node.js และ Express

ใช้พัฒนาบริการข้อมูลหลัก (โฟลเดอร์ `backend`) ให้บริการ REST API ภายใต้เส้นทาง `/api` สำหรับจัดการข้อมูลสถานที่ กิจกรรม ฐานความรู้ QR Code ของรางวัล พอยท์ ทริป และระบบสมาชิก พร้อมทั้งเชื่อมต่อฐานข้อมูลผ่านไลบรารี knex

1.2.3 Python และ FastAPI

ใช้พัฒนา AI Service (โฟลเดอร์ `chatbot-service`) ซึ่งรับผิดชอบแชตบอต ระบบวางแผนทริป การสกัดข้อมูลกิจกรรมจากข้อความโพสต์ และการสร้าง Embedding โดยเรียกโมเดลภาษาขนาดใหญ่ผ่าน KKU AI Gateway

1.2.4 PostgreSQL บน Supabase

ใช้เป็นฐานข้อมูลหลักของระบบ พร้อมส่วนขยาย pgvector สำหรับการค้นคืนข้อมูลเชิงเวกเตอร์ โดยโครงสร้างตารางอยู่ในไฟล์ migration ของโฟลเดอร์ `backend/supabase/migrations`

1.2.5 บริการภายนอก

ระบบใช้ Cloudinary สำหรับเก็บรูปภาพ Google Places API สำหรับนำเข้าข้อมูลสถานที่ Google Maps JavaScript API สำหรับแสดงแผนที่ และ Gmail SMTP สำหรับส่งอีเมลยืนยันและรีเซ็ตรหัสผ่าน

1.2.6 Visual Studio Code

ใช้เป็นเครื่องมือสำหรับเขียนและแก้ไขซอร์สโค้ดของโปรแกรม

1.3 ขอบเขตและข้อจำกัด

1.3.1 ขอบเขตของผู้ใช้งานทั่วไป (Guest)

(1) เข้าชมหน้าเว็บสาธารณะ ค้นหาและดูรายละเอียดสถานที่ท่องเที่ยว ร้านอาหาร คาเฟ่ และกิจกรรมได้

(2) สอบถามข้อมูลการท่องเที่ยวกับน้องไดโน (แชตบอต) ได้

(3) สร้างแผนการเดินทางด้วย AI ได้ แต่แผนจะสูญหายเมื่อปิดหน้าเว็บ เนื่องจากไม่ได้บันทึกลงบัญชี

(4) ดูรายการสถานที่ร่วมรายการและของรางวัลในหน้าพอยท์สะสมได้ แต่สะสมพอยท์ไม่ได้

1.3.2 ขอบเขตของสมาชิก (Tourist)

(1) สมัครสมาชิกด้วยอีเมล ยืนยันอีเมล และเข้าสู่ระบบ

(2) ใช้งานได้ทุกฟังก์ชันของผู้ใช้งานทั่วไป และบันทึกแผนทริปไว้ในเมนู “ทริปของฉัน”

(3) สแกน QR Code ณ สถานที่ร่วมรายการเพื่อสะสมพอยท์ และดูประวัติพอยท์

(4) แลกของรางวัลที่เคาน์เตอร์โดยแจ้งเจ้าหน้าที่

(5) แก้ไขข้อมูลโปรไฟล์ เปลี่ยนรหัสผ่านหรืออีเมล ส่งกิจกรรมใหม่เข้าระบบ และแจ้งข้อมูลสถานที่หรือกิจกรรมที่ไม่ถูกต้อง

1.3.3 ขอบเขตของผู้ดูแลระบบ (Admin)

(1) เข้าสู่ระบบผ่านหน้าเข้าสู่ระบบสำหรับผู้ดูแลระบบ

(2) ดูภาพรวมของระบบผ่านแดชบอร์ด

(3) เพิ่ม แก้ไข และลบข้อมูลสถานที่ กิจกรรม และฐานความรู้ของแชตบอต รวมถึงปรับปรุงข้อมูลสถานที่ให้ตรงกับ Google Places

(4) ตรวจสอบรายงานข้อมูลที่ไม่ถูกต้องและคำขอกิจกรรมจากผู้ใช้

(5) จัดการ QR Code ของรางวัล และดำเนินการแลกของรางวัลที่เคาน์เตอร์

(6) จัดการผู้ใช้ ดูทริปที่ผู้ใช้สร้าง และดูผลความพึงพอใจ

1.3.4 ข้อจำกัดของระบบ

(1) ข้อมูลสถานที่ครอบคลุมจังหวัดขอนแก่น โดยข้อมูลหลักมาจาก Google Places API และข้อมูลที่ผู้ดูแลระบบเพิ่มเอง จึงอาจไม่เป็นปัจจุบันหากยังไม่ได้ปรับปรุงข้อมูล

(2) แชตบอตและระบบวางแผนทริปใช้โมเดลภาษาขนาดใหญ่ แม้จำกัดการตอบให้อ้างอิงฐานความรู้ แต่ยังมีโอกาสตอบคลาดเคลื่อน และอาจใช้งานไม่ได้ชั่วคราวเมื่อโควตาของ KKU AI Gateway หมด

(3) QR Code แต่ละจุดให้พอยท์ได้ครั้งเดียวต่อบัญชี และการแลกของรางวัลต้องทำที่เคาน์เตอร์โดยเจ้าหน้าที่ ระบบยังไม่รองรับการแลกของรางวัลด้วยตนเอง

(4) ผู้ดูแลระบบเป็นบัญชีที่สร้างโดยนักพัฒนาผ่านสคริปต์ ไม่สามารถสมัครผ่านหน้าเว็บไซต์ได้

(5) ระบบเป็นเว็บแอปพลิเคชันที่ใช้ผ่านเบราว์เซอร์ ไม่มีแอปพลิเคชันบนมือถือแยกต่างหาก และการสแกน QR Code ต้องอนุญาตให้เว็บเข้าถึงกล้อง

บทที่ 2
คู่มือการใช้งาน

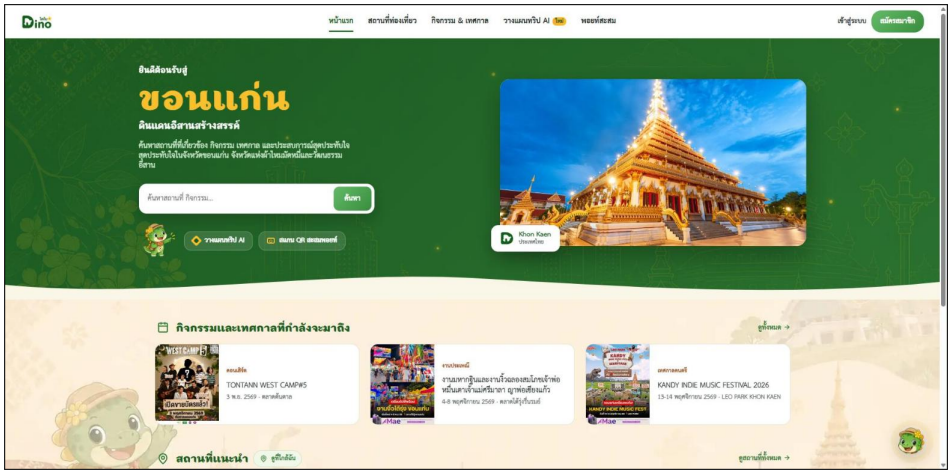
2.1 คู่มือการใช้งาน

“ไดโน” เว็บแอปพลิเคชันศูนย์กลางของการท่องเที่ยวจังหวัดขอนแก่น เข้าใช้งานได้ที่ https://dinokhonkaen.app แบ่งผู้ใช้งานออกเป็น 3 กลุ่ม ได้แก่

1. ผู้ใช้งานทั่วไป (Guest)
2. สมาชิก (Tourist)
3. ผู้ดูแลระบบ (Admin)

2.1.1 เข้าใช้งานกรณีเป็นผู้ใช้งานทั่วไป

แถบเมนูผู้ใช้งานทั่วไป อยู่ด้านบนของทุกหน้า ดังภาพที่ 1



ภาพที่ 1 หน้าแรกและแถบเมนูผู้ใช้งานทั่วไป

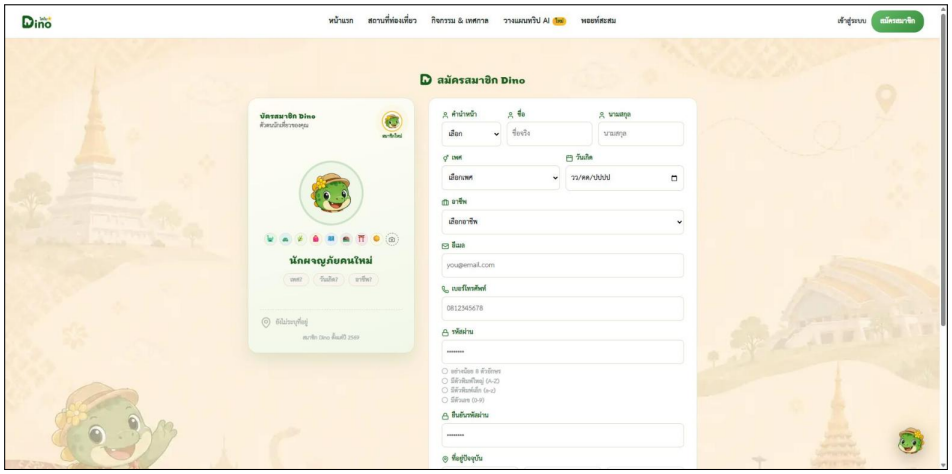
จากภาพที่ 1 รายละเอียดแถบเมนูมีดังนี้

1. หน้าแรก แสดงช่องค้นหา กิจกรรมและเทศกาลที่กำลังจะมาถึง และสถานที่แนะนำ
2. สถานที่ท่องเที่ยว แสดงรายการสถานที่ทั้งหมด พร้อมตัวกรองตามหมวดหมู่
3. กิจกรรม & เทศกาล แสดงรายการกิจกรรมและเทศกาลในขอนแก่น
4. วางแผนทริป AI ใช้สร้างแผนการเดินทางให้อัตโนมัติ
5. พอยท์สะสม แสดงสถานที่ร่วมรายการและของรางวัล
6. เข้าสู่ระบบ และสมัครสมาชิก อยู่มุมขวาบน

เมื่อ เปิด เว็บไซต์ ครั้ง แรก ระบบ อาจ แสดง หน้าต่าง ต้อนรับ จาก น้อง ไดโน สามารถกดปุ่ม “เริ่มสำรวจขอนแก่น” หรือ “ไม่แสดงอีก 7 วัน” เพื่อปิดได้ และมีปุ่มรูปน้องไดโนมุมขวาล่างของทุกหน้าสำหรับเปิดแชตบอต

(1) เมนูสมัครสมาชิก กรณีเป็นผู้ใช้งานทั่วไป

การสมัครสมาชิกทำได้โดยเลือกปุ่ม “สมัครสมาชิก” มุมขวาบนของหน้าเว็บ ระบบจะแสดงแบบฟอร์มสมัครสมาชิกดังภาพที่ 2



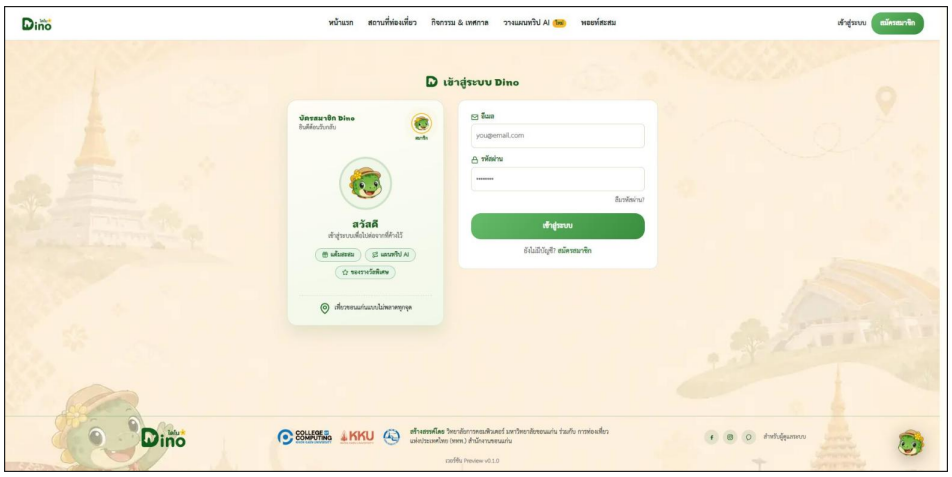
ภาพที่ 2 หน้าฟอร์มสมัครสมาชิก

จากภาพที่ 2 ผู้ใช้กรอกข้อมูล ได้แก่ คำนำหน้า ชื่อ นามสกุล เพศ วันเกิด อาชีพ อีเมล เบอร์โทรศัพท์ รหัสผ่านและยืนยันรหัสผ่าน และที่อยู่ปัจจุบัน (จังหวัด อำเภอ ตำบล) โดยรหัสผ่านต้องมีความยาวอย่างน้อย 8 ตัวอักษร ประกอบด้วยตัวพิมพ์ใหญ่ ตัวพิมพ์เล็ก และตัวเลข ระบบแสดงผลการตรวจสอบเงื่อนไขทีละข้อใต้ช่องรหัสผ่าน จากนั้นต้องทำเครื่องหมายยินยอมให้เก็บและใช้ข้อมูลส่วนบุคคลตามนโยบายความเป็นส่วนตัว แล้วกดปุ่ม “สมัครสมาชิก”

หลังสมัครสำเร็จ ระบบจะส่งอีเมลยืนยันไปยังที่อยู่อีเมลที่กรอก ผู้ใช้ต้องเลือกลิงก์ในอีเมลเพื่อยืนยันบัญชีก่อน จึงจะเข้าสู่ระบบได้ หากพยายามเข้าสู่ระบบก่อนยืนยัน ระบบจะแจ้งว่า “กรุณายืนยันอีเมลก่อนเข้าสู่ระบบ” และเมื่อยืนยันสำเร็จ ระบบจะเข้าสู่ระบบให้โดยอัตโนมัติ

(2) เมนูเข้าสู่ระบบ กรณีเป็นผู้ใช้งานทั่วไป

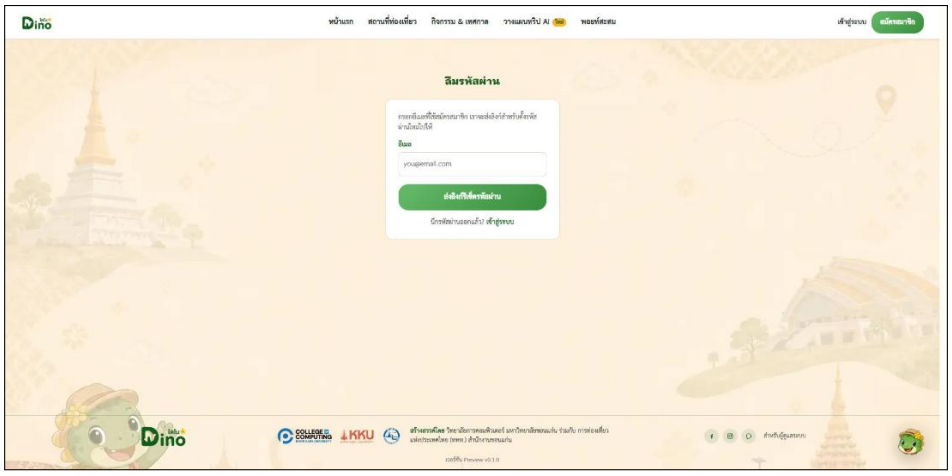
ผู้ใช้ที่มีบัญชีแล้วเลือกเมนู “เข้าสู่ระบบ” มุมขวาบน ระบบจะแสดงแบบฟอร์มเข้าสู่ระบบดังภาพที่ 3



ภาพที่ 3 หน้าฟอร์มเข้าสู่ระบบ

ผู้ใช้กรอกอีเมลและรหัสผ่านที่ลงทะเบียนไว้ แล้วกดปุ่ม “เข้าสู่ระบบ” หากยังไม่มีบัญชีสามารถเลือก “สมัครสมาชิก” ด้านล่างปุ่ม เพื่อไปยังหน้าสมัครสมาชิกได้ทันที

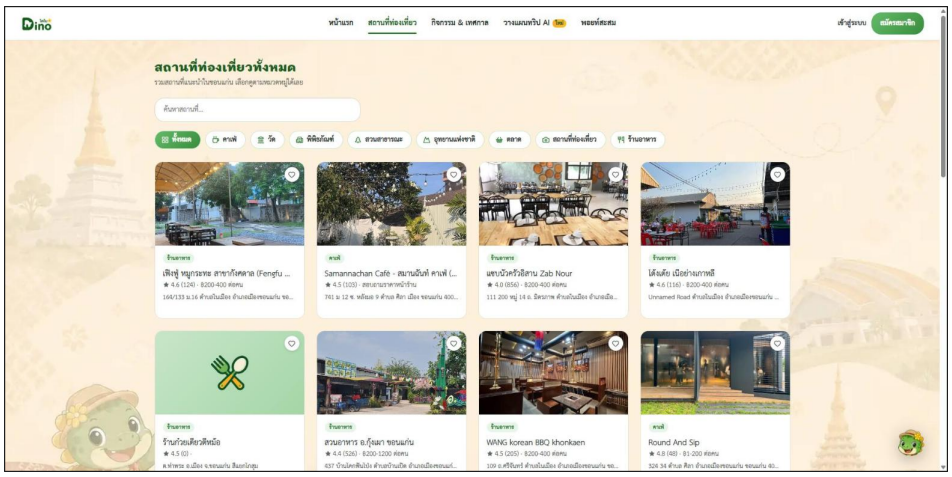
หากลืมรหัสผ่าน ให้เลือก “ลืมรหัสผ่าน?” เพื่อเข้าสู่หน้าดังภาพที่ 4 กรอกอีเมลที่ใช้สมัครแล้วกด “ส่งลิงก์รีเซ็ตรหัสผ่าน” ระบบจะส่งลิงก์สำหรับตั้งรหัสผ่านใหม่ไปยังอีเมลนั้น



ภาพที่ 4 หน้าลืมรหัสผ่าน

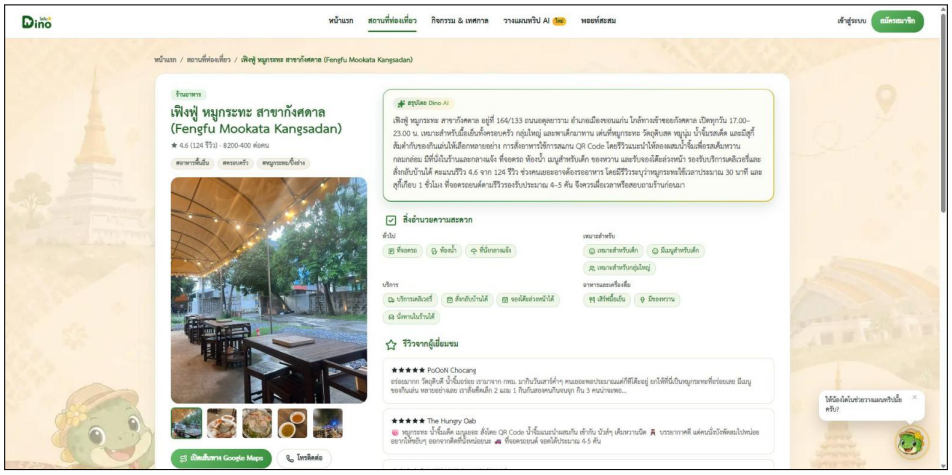
(3) เมนูสถานที่ท่องเที่ยว กรณีเป็นผู้ใช้งานทั่วไป

ผู้ใช้เรียกดูสถานที่ได้จากเมนู “สถานที่ท่องเที่ยว” ระบบจะแสดงรายการสถานที่ในรูปแบบการ์ด พร้อมช่องค้นหาและตัวกรองหมวดหมู่ ได้แก่ ทั้งหมด คาเฟ่ วัด พิพิธภัณฑ์ สวนสาธารณะ อุทยานแห่งชาติ ตลาด สถานที่ท่องเที่ยว และร้านอาหาร ดังภาพที่ 5



ภาพที่ 5 หน้ารายการสถานที่ท่องเที่ยว

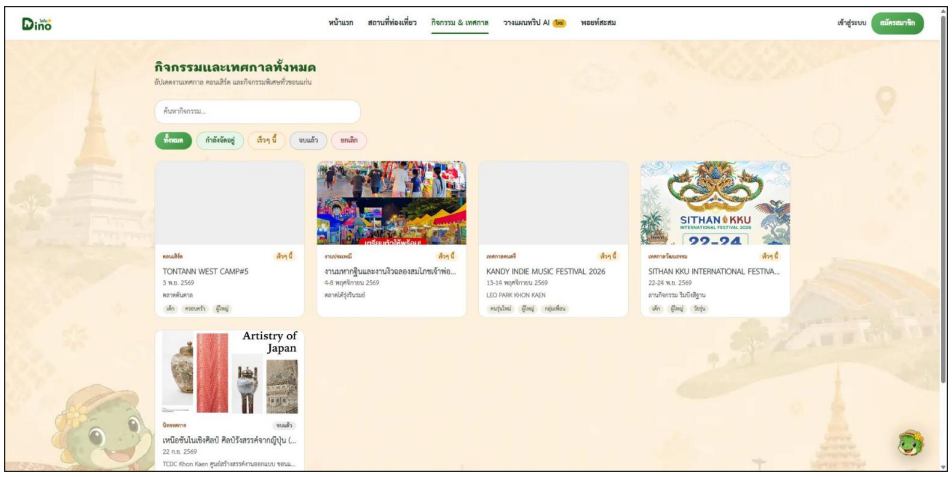
เมื่อเลือกการ์ดสถานที่ ระบบจะแสดงหน้ารายละเอียดดังภาพที่ 6 ซึ่งประกอบด้วยหมวดหมู่ คะแนนและช่วงราคา แกลเลอรีรูปภาพ สรุปโดย Dino AI สิ่งอำนวยความสะดวก และรีวิวจากผู้เยี่ยมชม พร้อมปุ่ม “เปิดเส้นทาง Google Maps” และปุ่ม “โทรติดต่อ”



ภาพที่ 6 หน้ารายละเอียดสถานที่ท่องเที่ยว

(4) เมนูกิจกรรมและเทศกาล กรณีเป็นผู้ใช้งานทั่วไป

เมนู “กิจกรรม & เทศกาล” แสดงรายการกิจกรรมและเทศกาลในขอนแก่น พร้อมช่องค้นหาและตัวกรองสถานะ ได้แก่ ทั้งหมด กำลังจัดอยู่ เร็ว ๆ นี้ จบแล้ว และยกเลิก ดังภาพที่ 7



ภาพที่ 7 หน้ารายการกิจกรรมและเทศกาล

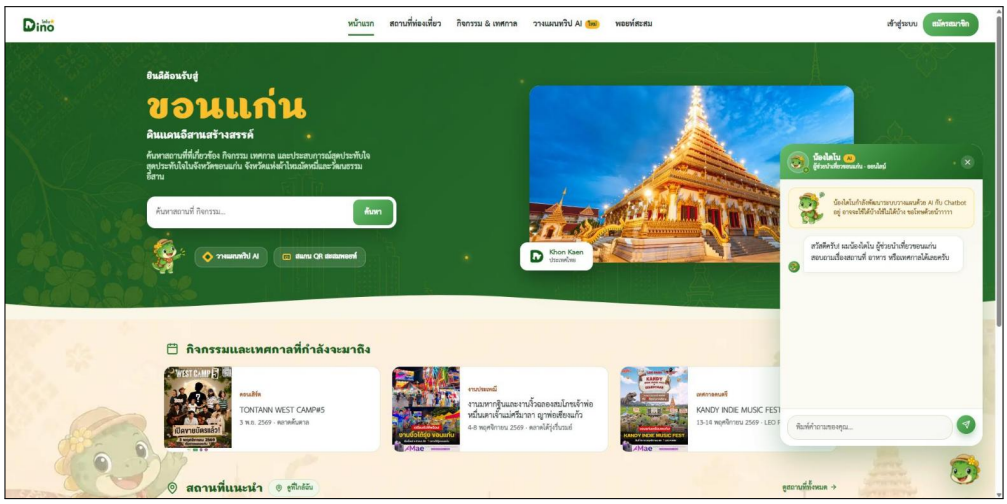
เมื่อเลือกกิจกรรม ระบบจะแสดงรายละเอียด ได้แก่ รูปภาพ คำอธิบาย วันจัดงาน สถานที่จัดงาน ค่าเข้าชม ผู้จัดงาน กลุ่มที่เหมาะสม และแผนที่ ดังภาพที่ 8 หากพบข้อมูลไม่ถูกต้อง สามารถกดปุ่ม “แจ้งข้อมูลไม่ถูกต้อง” ใต้รูปภาพเพื่อส่งให้ผู้ดูแลระบบตรวจสอบได้



ภาพที่ 8 หน้ารายละเอียดกิจกรรม

(5) เมนูแชตบอต กรณีเป็นผู้ใช้งานทั่วไป

ผู้ใช้เลือกปุ่มรูปน้องไดโนมุมขวาล่างเพื่อเปิดหน้าต่างแชตบอต พิมพ์คำถามเกี่ยวกับสถานที่ อาหาร หรือเทศกาลในช่องด้านล่าง แล้วกดปุ่มส่ง ดังภาพที่ 9

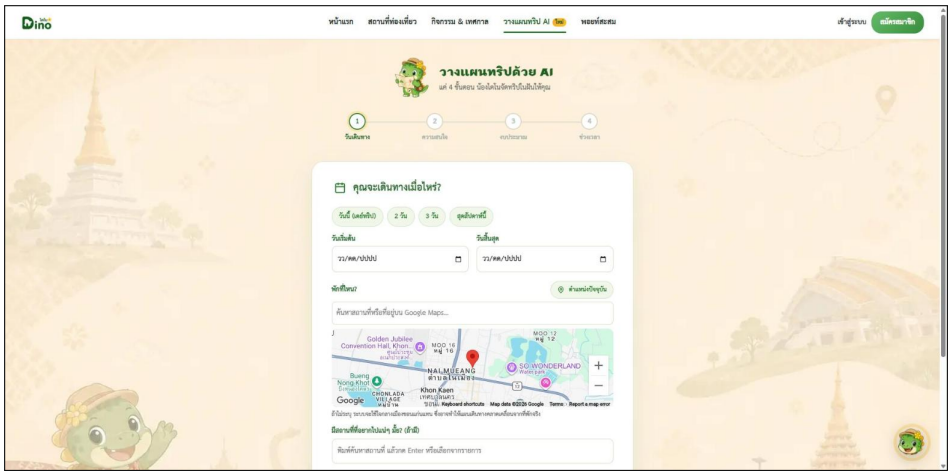


ภาพที่ 9 หน้าต่างแชตบอตน้องไดโน

ระบบจะค้นหาข้อมูลที่เกี่ยวข้องจากฐานความรู้และแสดงคำตอบต่อเนื่องทีละส่วน หากไม่พบข้อมูลที่เกี่ยวข้อง แชตบอตจะแจ้งว่าไม่มีข้อมูลแทนการตอบโดยไม่มีข้อมูลรองรับ ในกรณีที่ปรากฏข้อความ “ระบบแชทขัดข้องชั่วคราว” ให้ดำเนินการอีกครั้งในภายหลัง

(6) เมนูวางแผนทริป AI กรณีเป็นผู้ใช้งานทั่วไป

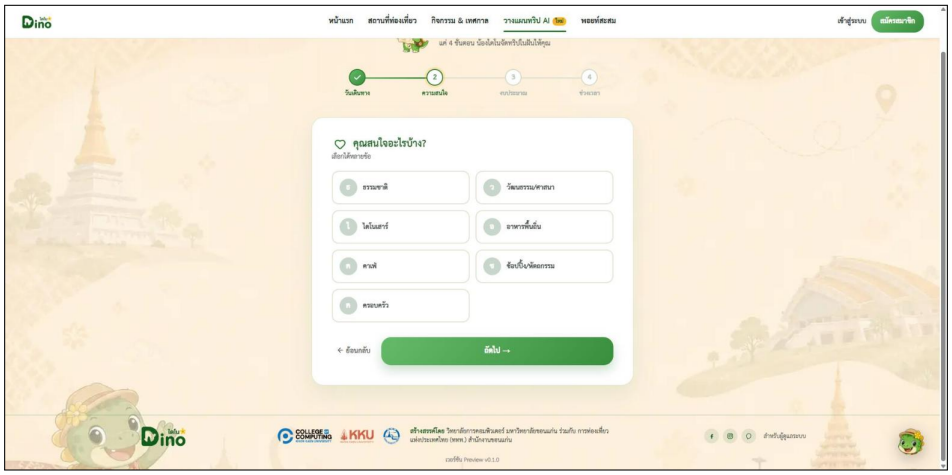
ผู้ใช้เลือกเมนู “วางแผนทริป AI” เพื่อสร้างแผนการเดินทางผ่านแบบฟอร์ม 4 ขั้นตอน โดยมีแถบแสดงความคืบหน้าด้านบน ขั้นตอนที่ 1 วันเดินทาง ดังภาพที่ 10



ภาพที่ 10 หน้าแบบฟอร์มสร้างแผนการเดินทาง ขั้นตอนที่ 1 วันเดินทาง

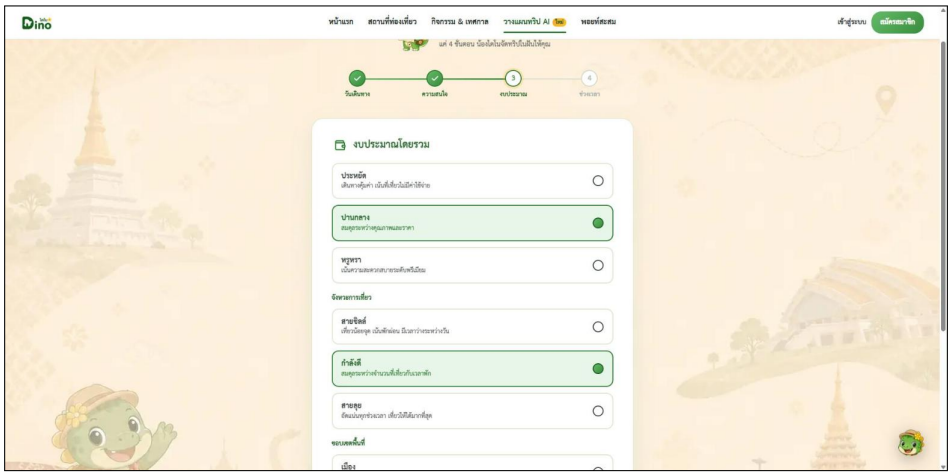
จากภาพที่ 10 ผู้ใช้เลือกช่วงเวลาเดินทางจากปุ่มลัด ได้แก่ วันนี้ (ทริปแบบไปเช้า-เย็นกลับ) 2 วัน 3 วัน หรือสุดสัปดาห์นี้ หรือกำหนดวันเริ่มต้นและวันสิ้นสุดเอง ระบุที่พัก โดยพิมพ์ค้นหาหรือกด “ตำแหน่งปัจจุบัน” และระบุสถานที่ที่ต้องการไปแน่นอน (ถ้ามี) หากไม่ระบุที่พัก ระบบจะใช้ใจกลางเมืองขอนแก่นแทน จากนั้นกด “ถัดไป”

ขั้นตอนที่ 2 ความสนใจ ผู้ใช้เลือกได้หลายข้อ ได้แก่ ธรรมชาติ วัฒนธรรม/ศาสนา ไดโนเสาร์ อาหารพื้นถิ่น คาเฟ่ ช้อปปิ้ง/หัตถกรรม และครอบครัว ดังภาพที่ 11



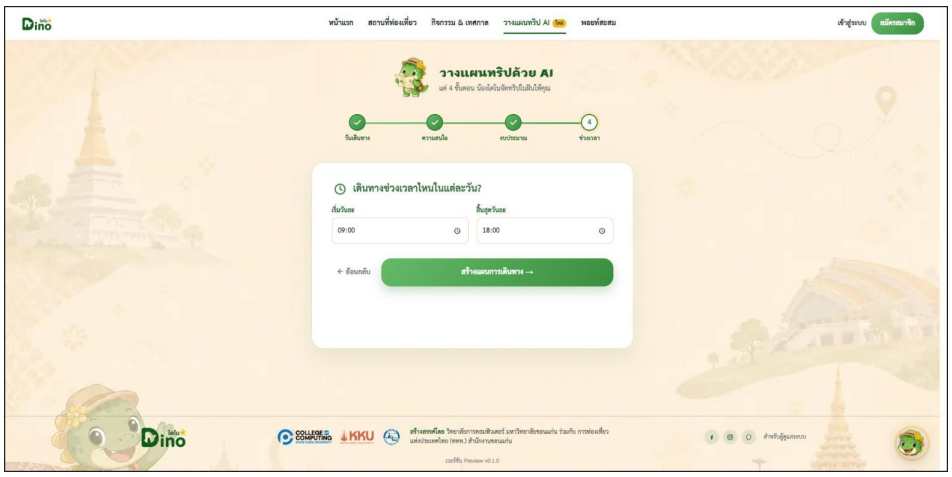
ภาพที่ 11 หน้าแบบฟอร์มสร้างแผนการเดินทาง ขั้นตอนที่ 2 ความสนใจ

ขั้นตอนที่ 3 งบประมาณ ผู้ใช้เลือกงบประมาณโดยรวม (ประหยัด ปานกลาง หรูหรา) จังหวะการเที่ยว (สายชิลล์ กำลังดี หรือสายลุย) และขอบเขตพื้นที่ (เมือง หรือทั่วขอนแก่นซึ่งรวมสถานที่รอบนอก เช่น ภูเวียง อุบลรัตน์) ดังภาพที่ 12



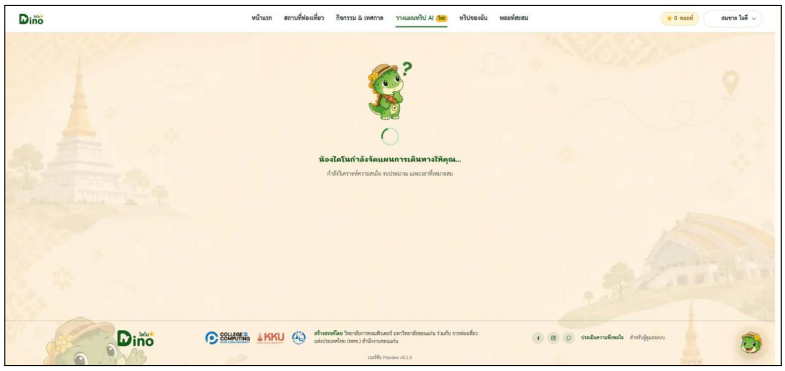
ภาพที่ 12 หน้าแบบฟอร์มสร้างแผนการเดินทาง ขั้นตอนที่ 3 งบประมาณ

ขั้นตอนที่ 4 ช่วงเวลา ผู้ใช้กำหนดเวลาเริ่มและเวลาสิ้นสุดของแต่ละวัน (ค่าเริ่มต้น 09:00 ถึง 18:00) แล้วกดปุ่ม “สร้างแผนการเดินทาง” ดังภาพที่ 13

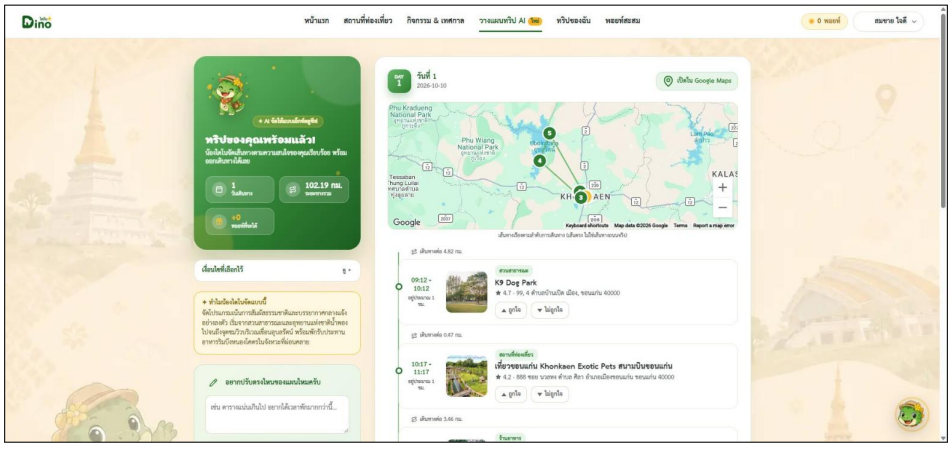


ภาพที่ 13 หน้าแบบฟอร์มสร้างแผนการเดินทาง ขั้นตอนที่ 4 ช่วงเวลา

เมื่อกดสร้างแผน ระบบแสดงหน้าสถานะกำลังประมวลผล ดังภาพที่ 14 จากนั้นแสดงแผนการเดินทางรายวันพร้อมแผนที่เส้นทาง ดังภาพที่ 15



ภาพที่ 14 หน้าสถานะกำลังประมวลผลแผนการเดินทาง

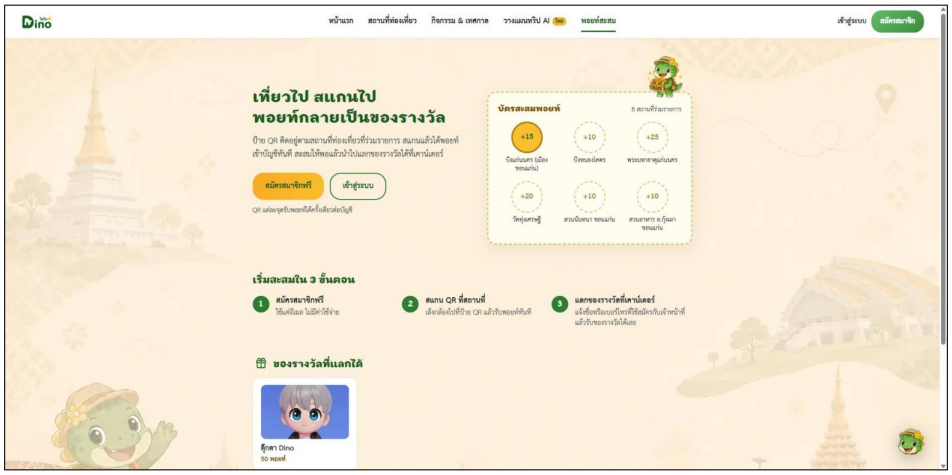


ภาพที่ 15 หน้าแสดงผลแผนการเดินทาง

ในหน้าผลลัพธ์ ผู้ใช้สามารถพิมพ์สิ่งที่อยากปรับ แล้วกดปุ่ม “ปรับแผนการเดินทางอีกครั้ง” เพื่อให้ระบบสร้างแผนใหม่ ทั้งนี้ ผู้ใช้งานทั่วไปที่ไม่ได้เข้าสู่ระบบจะไม่สามารถบันทึกแผนได้ และแผนจะสูญหายเมื่อปิดหน้าเว็บ จึงควรเข้าสู่ระบบก่อนสร้างแผนเพื่อบันทึกไว้ในเมนู “ทริปของฉัน”

(7) เมนูพอยท์สะสม กรณีเป็นผู้ใช้งานทั่วไป

เมนู “พอยท์สะสม” สำหรับผู้ที่ยังไม่เข้าสู่ระบบ แสดงบัตรสะสมพอยท์ที่ระบุสถานที่ร่วมรายการพร้อมจำนวนพอยท์ที่ได้รับจากแต่ละที่ ขั้นตอนการสะสม 3 ขั้นตอน และรายการของรางวัลที่แลกได้ ดังภาพที่ 16

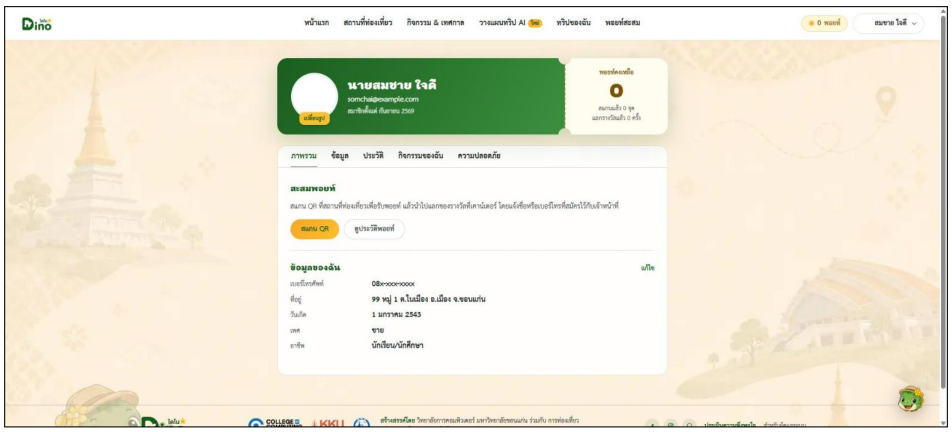


ภาพที่ 16 หน้าพอยท์สะสม กรณีผู้ใช้งานทั่วไป

ผู้ใช้สามารถเลือก “สมัครสมาชิกฟรี” หรือ “เข้าสู่ระบบ” เพื่อเริ่มสะสมพอยท์ได้

2.1.2 เข้าใช้งานกรณีเข้าสู่ระบบของสมาชิก

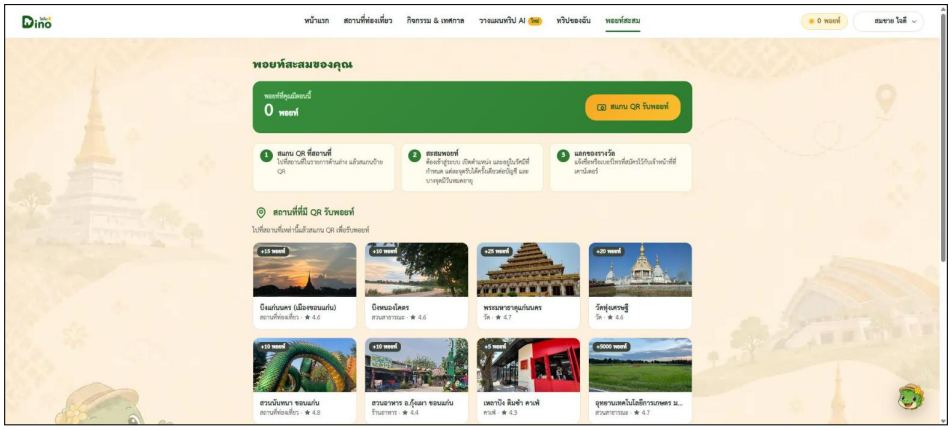
เมื่อเข้าสู่ระบบแล้ว แถบเมนูจะเพิ่มเมนู “ทริปของฉัน” และแทนที่ปุ่มเข้าสู่ระบบด้วยเมนูผู้ใช้มุมขวาบน ดังภาพที่ 17



ภาพที่ 17 แถบเมนูสมาชิก

(1) เมนูพอยท์สะสม กรณีเป็นสมาชิก

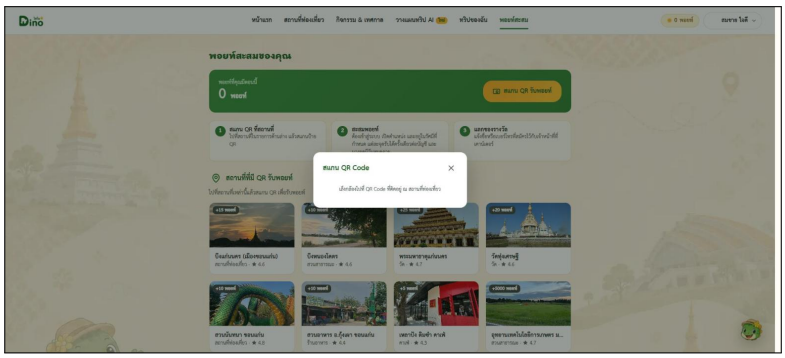
หน้า “พอยท์สะสมของคุณ” แสดงยอดพอยท์ที่มีอยู่ ปุ่มสแกน QR Code รายการสถานที่ที่มี QR สำหรับรับพอยท์ และรายการของรางวัล ดังภาพที่ 18



ภาพที่ 18 หน้าพอยท์สะสมของสมาชิก

(2) การสแกน QR Code เพื่อสะสมพอยท์ กรณีเป็นสมาชิก

เมื่อไปถึงสถานที่ร่วมรายการ สมาชิกสแกนป้าย QR Code ด้วยปุ่มสแกนในหน้าพอยท์สะสม หรือใช้กล้องของโทรศัพท์มือถือสแกนโดยตรงก็ได้ ระบบจะขออนุญาตใช้กล้อง ดังภาพที่ 19



ภาพที่ 19 หน้าต่างสแกน QR Code

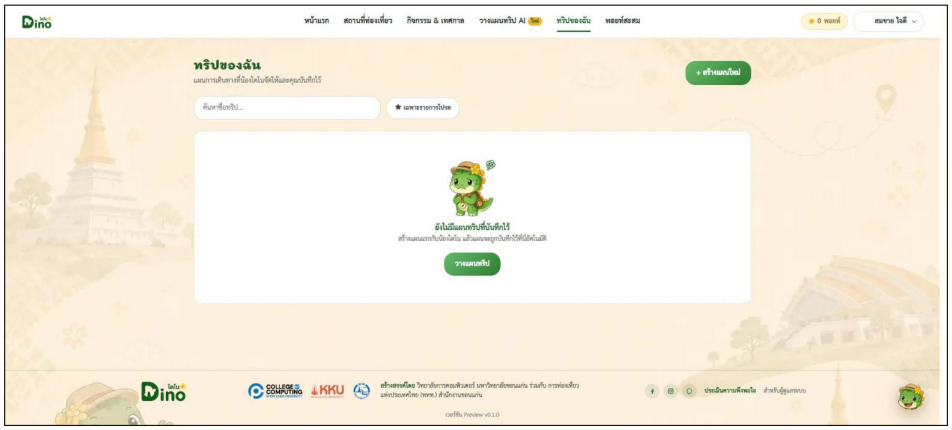
เมื่อสแกนสำเร็จ ระบบเพิ่มพอยท์เข้าบัญชีทันทีและแสดงผลการสแกน QR Code แต่ละจุดรับพอยท์ได้ครั้งเดียวต่อบัญชี หากสแกนซ้ำหรือ QR Code ไม่ถูกต้อง ระบบจะแสดงข้อความ “สแกนไม่สำเร็จ” พร้อมเหตุผล

(3) การแลกของรางวัล กรณีเป็นสมาชิก

การแลกของรางวัลดำเนินการที่เคาน์เตอร์ โดยสมาชิกแจ้งชื่อหรือเบอร์โทรศัพท์ที่ใช้สมัครกับเจ้าหน้าที่ เจ้าหน้าที่จะหักพอยท์และมอบของรางวัลผ่านหน้าผู้ดูแลระบบตามหัวข้อที่ 2.1.3

(4) เมนูทริปของฉัน กรณีเป็นสมาชิก

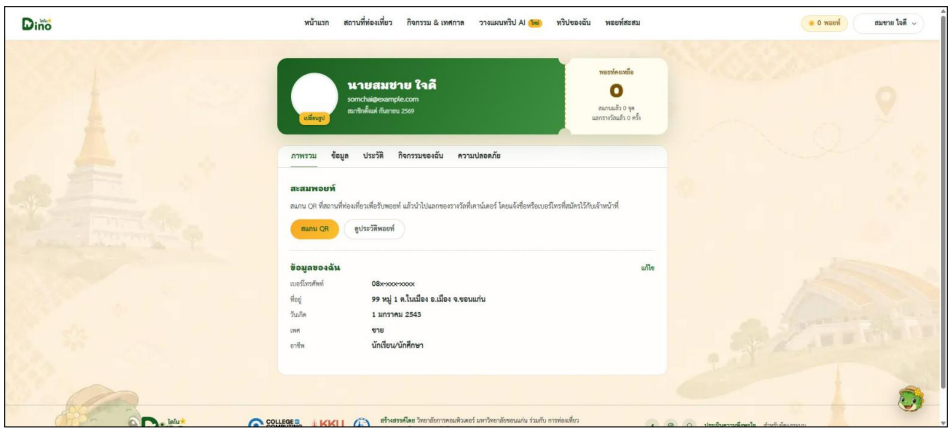
เมนู “ทริปของฉัน” แสดงแผนทริปที่บันทึกไว้ทั้งหมด ดังภาพที่ 20 ผู้ใช้สามารถค้นหาตามชื่อทริป เปลี่ยนชื่อ เพิ่มในรายการโปรด ทำสำเนา และลบแผนได้ รวมถึงกด “+ สร้างแผนใหม่” เพื่อสร้างแผนเพิ่ม



ภาพที่ 20 หน้าทริปของฉัน

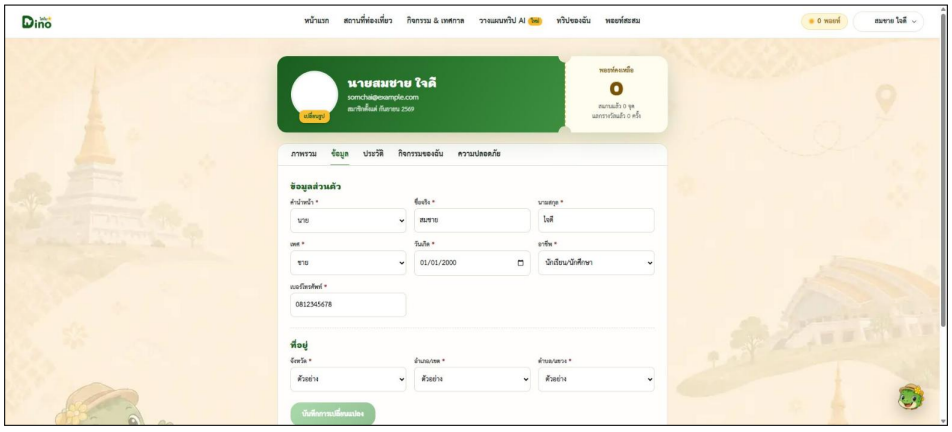
(5) เมนูโปรไฟล์ กรณีเป็นสมาชิก

ผู้ใช้เปิดหน้าโปรไฟล์ได้จากเมนูผู้ใช้มุมขวาบน ประกอบด้วยแท็บ 5 แท็บ ได้แก่ ภาพรวม ข้อมูล ประวัติ กิจกรรมของฉัน และความปลอดภัย ดังภาพที่ 21

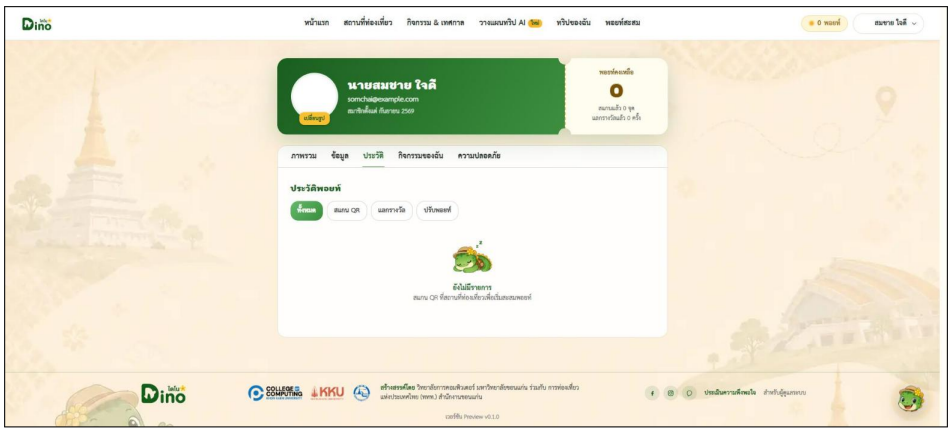


ภาพที่ 21 หน้าโปรไฟล์ แท็บภาพรวม

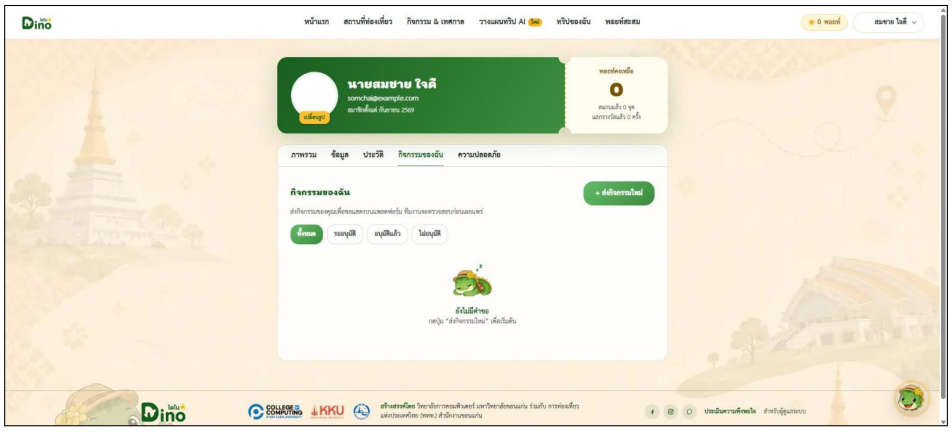
1. ภาพรวม แสดงข้อมูลสรุปของผู้ใช้ ยอดพอยท์ และปุ่มลัด “สแกน QR สะสมพอยท์” และ “ดูประวัติพอยท์”
2. ข้อมูล ใช้แก้ไขข้อมูลส่วนตัว เช่น คำนำหน้า ชื่อ นามสกุล วันเกิด อาชีพ เบอร์โทรศัพท์ และที่อยู่ แล้วกด “บันทึกการเปลี่ยนแปลง” ดังภาพที่ 22
3. ประวัติ แสดงประวัติพอยท์ทั้งการสแกน QR การแลกรางวัล และการปรับพอยท์ พร้อมตัวกรอง ดังภาพที่ 23
4. กิจกรรมของฉัน ใช้ส่งกิจกรรมใหม่เข้าระบบด้วยปุ่ม “+ ส่งกิจกรรมใหม่” และติดตามสถานะรออนุมัติ อนุมัติแล้ว หรือไม่อนุมัติ ดังภาพที่ 24
5. ความปลอดภัย ใช้เปลี่ยนรหัสผ่าน เปลี่ยนอีเมล และลบบัญชี ดังภาพที่ 25



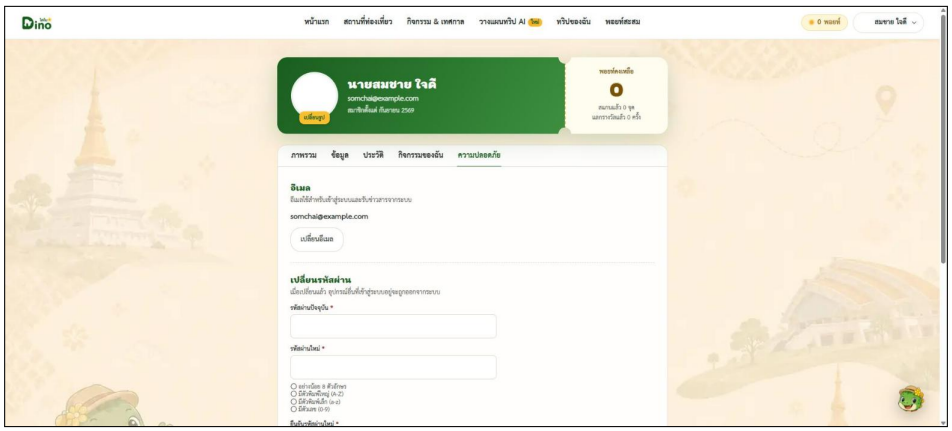
ภาพที่ 22 หน้าโปรไฟล์ แท็บข้อมูล



ภาพที่ 23 หน้าโปรไฟล์ แท็บประวัติ



ภาพที่ 24 หน้าโปรไฟล์ แท็บกิจกรรมของฉัน

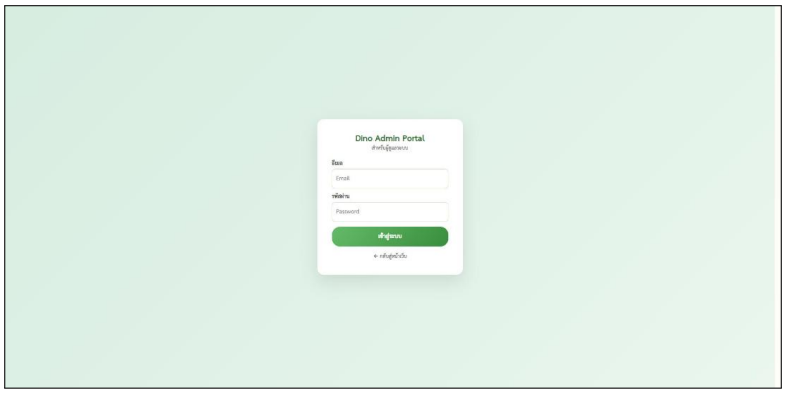


ภาพที่ 25 หน้าโปรไฟล์ แท็บความปลอดภัย

2.1.3 เข้าใช้งานกรณีเข้าสู่ระบบของผู้ดูแลระบบ

(1) เมนูเข้าสู่ระบบ กรณีเป็นผู้ดูแลระบบ

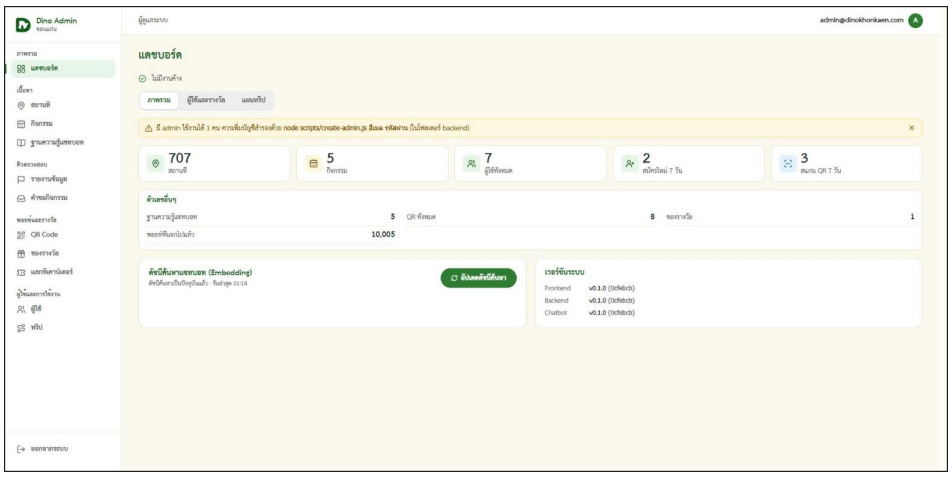
ผู้ดูแลระบบเข้าสู่ระบบที่ https://dinokhonkaen.app/admin/login หรือเลือกลิงก์ “สำหรับผู้ดูแลระบบ” ที่ส่วนท้ายของเว็บไซต์ ดังภาพที่ 26 จากนั้นกรอกอีเมลและรหัสผ่านของบัญชีผู้ดูแลระบบแล้วกด “เข้าสู่ระบบ” บัญชีผู้ดูแลระบบสร้างโดยนักพัฒนาตามหัวข้อที่ 3.2.10



ภาพที่ 26 หน้าเข้าสู่ระบบสำหรับผู้ดูแลระบบ

(2) แถบเมนูและแดชบอร์ด

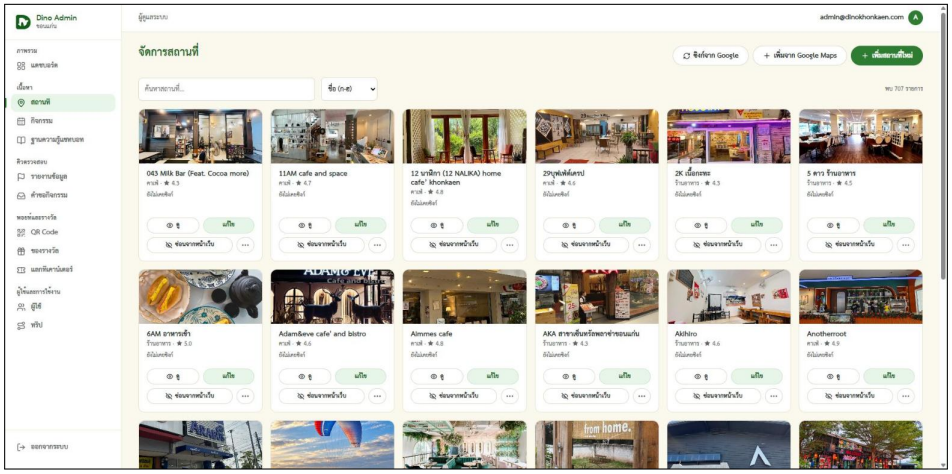
หลังเข้าสู่ระบบ ระบบแสดงแถบเมนูด้านข้างที่จัดเป็นกลุ่ม ดังนี้ ภาพรวม (แดชบอร์ด) เนื้อหา (สถานที่ กิจกรรม ฐานความรู้แชทบอท) คิวตรวจสอบ (รายงานข้อมูล คำขอกิจกรรม) พอยท์และรางวัล (QR Code ของรางวัล แลกที่เคาน์เตอร์) และผู้ใช้และการใช้งาน (ผู้ใช้ ทริป ความพึงพอใจ) ซึ่งชื่อเมนูตรงกับที่แสดงในระบบ ส่วนแดชบอร์ดแบ่งเป็น 3 แท็บ ได้แก่ ภาพรวม ผู้ใช้และรางวัล และแผนทริป โดยแท็บภาพรวมแสดงจำนวนสถานที่ กิจกรรม ผู้ใช้ทั้งหมด ผู้สมัครใหม่ และการสแกน QR Code ใน 7 วัน รวมถึงจำนวนฐานความรู้ QR Code ของรางวัล และพอยท์ที่แจกไปแล้ว นอกจากนี้ยังมีส่วนอัปเดตดัชนีค้นหาของแชตบอตและเวอร์ชันของระบบ ดังภาพที่ 27



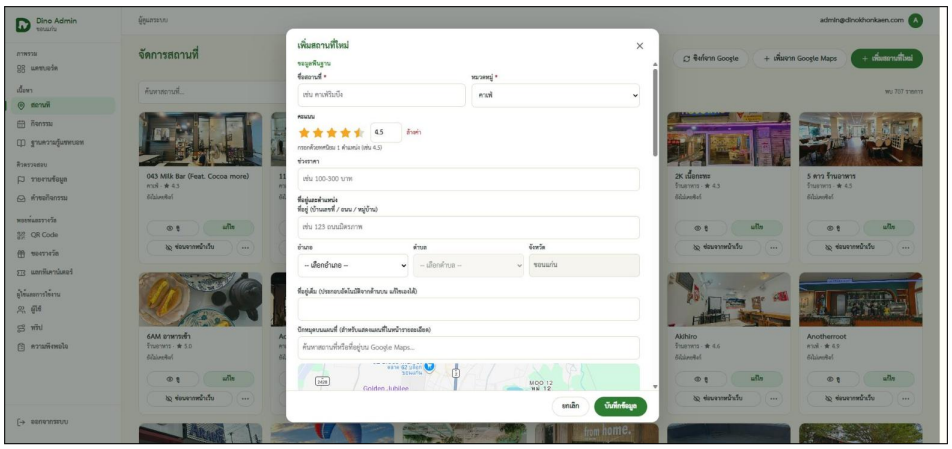
ภาพที่ 27 หน้าแดชบอร์ดผู้ดูแลระบบ

(3) เมนูสถานที่

ผู้ดูแลระบบจัดการข้อมูลสถานที่ได้จากเมนู “สถานที่” ดังภาพที่ 28 โดยเพิ่มสถานที่ใหม่ แก้ไขข้อมูล ลบสถานที่ และนำเข้าหรือปรับปรุงข้อมูลให้ตรงกับ Google Places ได้ ฟอร์มเพิ่มและแก้ไขข้อมูลสถานที่แสดงดังภาพที่ 29 ในการปรับปรุงข้อมูล ผู้ดูแลระบบสามารถล็อกช่องข้อมูลที่แก้ไขด้วยตนเองไว้ได้ ค่าใหม่จาก Google ของช่องข้อมูลที่ล็อกจะถูกพักไว้ให้ตรวจสอบก่อน และการปรับปรุงข้อมูลแต่ละสถานที่มีค่าใช้จ่ายตามการเรียกใช้ Google Places API



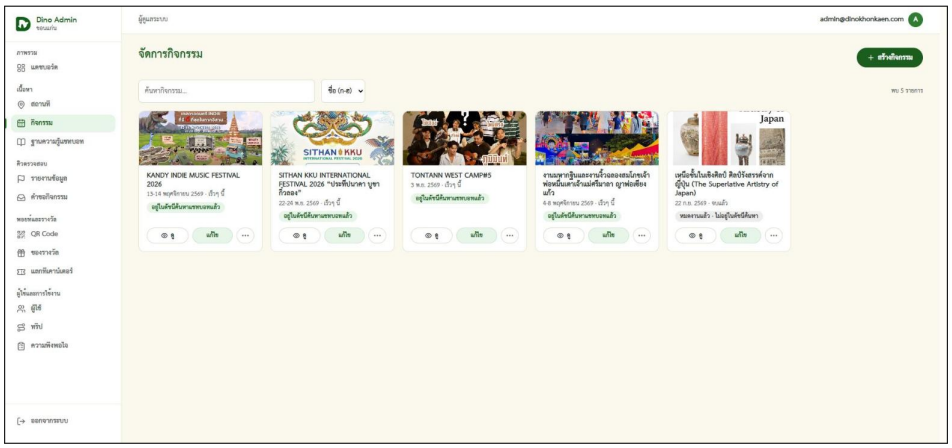
ภาพที่ 28 หน้าจัดการสถานที่



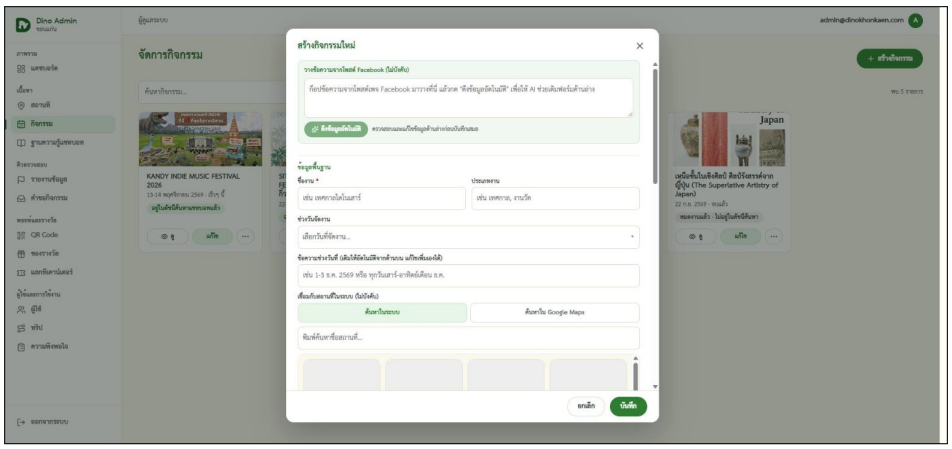
ภาพที่ 29 หน้าฟอร์มเพิ่มและแก้ไขข้อมูลสถานที่

(4) เมนูกิจกรรม

เมนู “กิจกรรม” ใช้เพิ่ม แก้ไข และลบกิจกรรมและเทศกาล ดังภาพที่ 30 ในฟอร์มกิจกรรมมีเครื่องมือช่วยสกัดข้อมูลจากข้อความโพสต์ด้วย AI โดยผู้ดูแลระบบวางข้อความโพสต์ แล้วระบบจะกรอกแบบฟอร์มให้โดยอัตโนมัติ ดังภาพที่ 31 ผู้ดูแลระบบควรตรวจสอบข้อมูลที่ AI กรอกให้ก่อนกดบันทึกทุกครั้ง



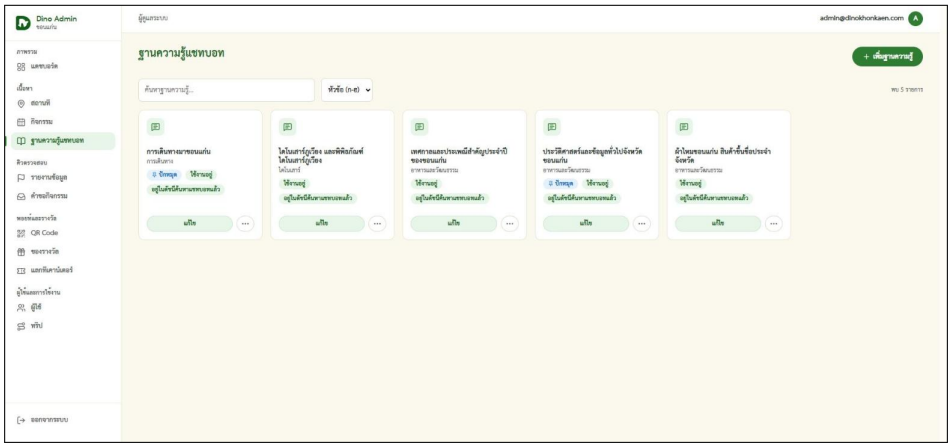
ภาพที่ 30 หน้าจัดการกิจกรรม



ภาพที่ 31 หน้าแบบฟอร์มกิจกรรมพร้อมเครื่องมือสกัดข้อมูลจากข้อความโพสต์ด้วย AI

(5) เมนูฐานความรู้แชตบอต

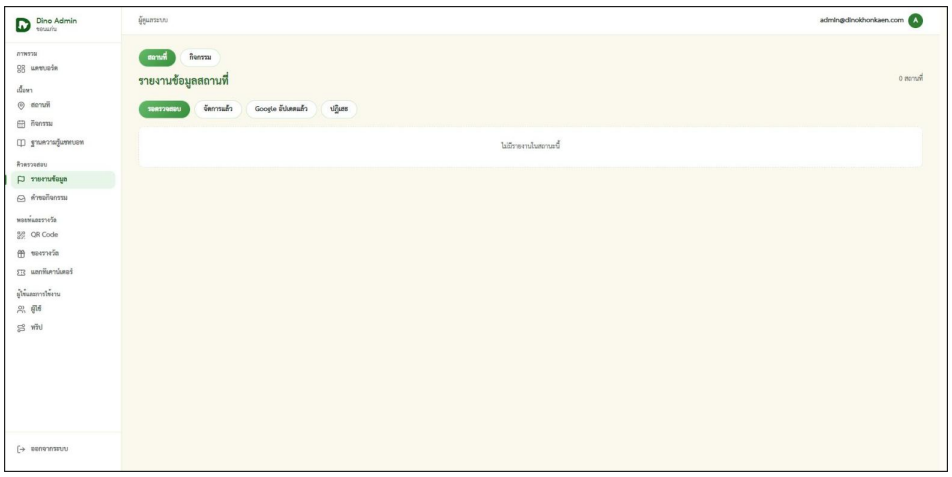
ผู้ดูแลระบบเพิ่ม แก้ไข ปักหมุด และเปิด/ปิดการใช้งานเอกสารความรู้ที่แชตบอตใช้อ้างอิง ดังภาพที่ 32 หลังเพิ่มหรือแก้ไขข้อมูล ต้องให้ระบบสร้าง Embedding ใหม่ เพื่อให้แชตบอตค้นหาข้อมูลล่าสุดได้ โดยกดปุ่ม “อัปเดตดัชนีค้นหา” ในการ์ด “ดัชนีค้นหาแชตบอท (Embedding)” บนหน้าแดชบอร์ด



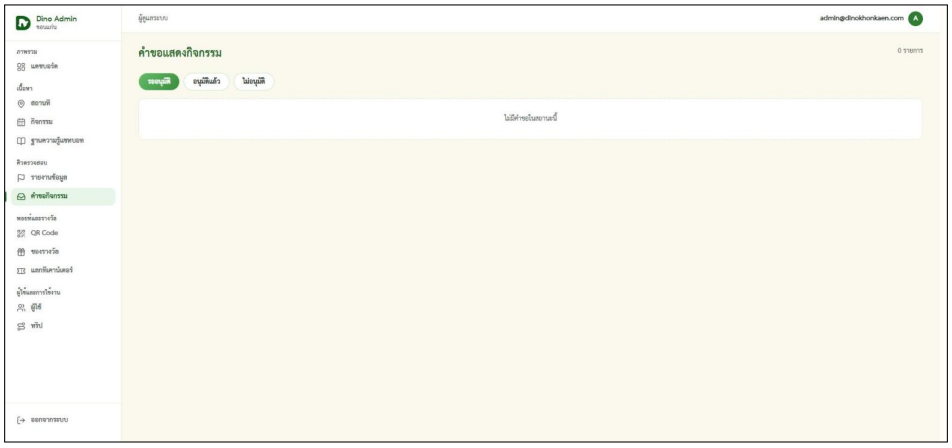
ภาพที่ 32 หน้าจัดการฐานความรู้แชตบอต

(6) เมนูรายงานข้อมูลและคำขอกิจกรรม

เมนู “รายงานข้อมูล” แสดงคิวรายงานข้อมูลสถานที่และกิจกรรมที่ผู้ใช้แจ้งว่าไม่ถูกต้อง ผู้ดูแลระบบตรวจสอบและดำเนินการ ดังภาพที่ 33 ส่วนเมนู “คำขอกิจกรรม” แสดงกิจกรรมที่สมาชิกส่งเข้ามา ผู้ดูแลระบบเลือก “อนุมัติ” เพื่อแสดงกิจกรรมบนหน้าเว็บ หรือ “ไม่อนุมัติ” พร้อมระบุเหตุผลที่ผู้ขอจะเห็น ดังภาพที่ 34



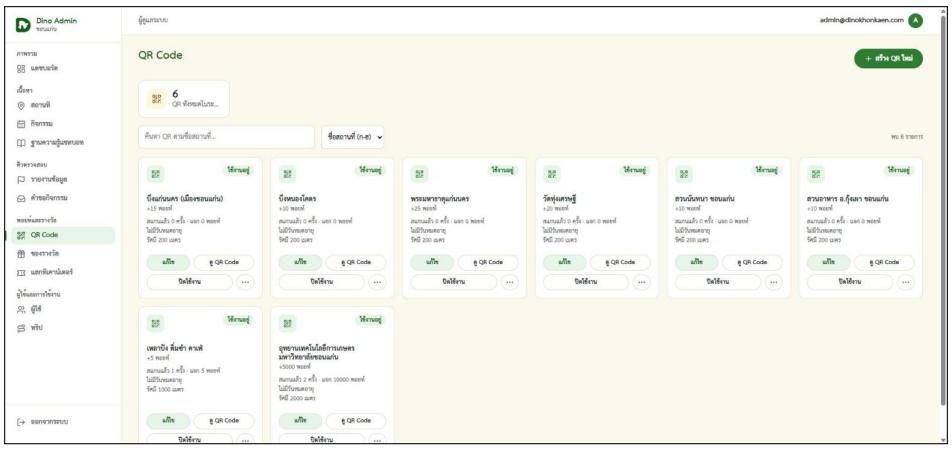
ภาพที่ 33 หน้ารายงานข้อมูล



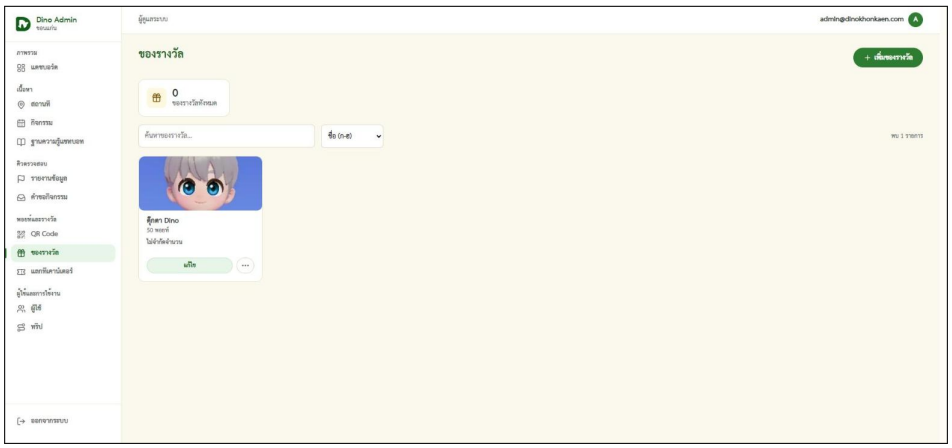
ภาพที่ 34 หน้าคำขอกิจกรรม

(7) เมนู QR Code และของรางวัล

เมนู “QR Code” ใช้สร้าง QR Code ผูกกับสถานที่และกำหนดจำนวนพอยท์ พร้อมดูตัวอย่างและพิมพ์ป้าย ดังภาพที่ 35 ส่วนเมนู “ของรางวัล” ใช้เพิ่มและแก้ไขรายการของรางวัล ราคาพอยท์ และจำนวนคงเหลือ ดังภาพที่ 36



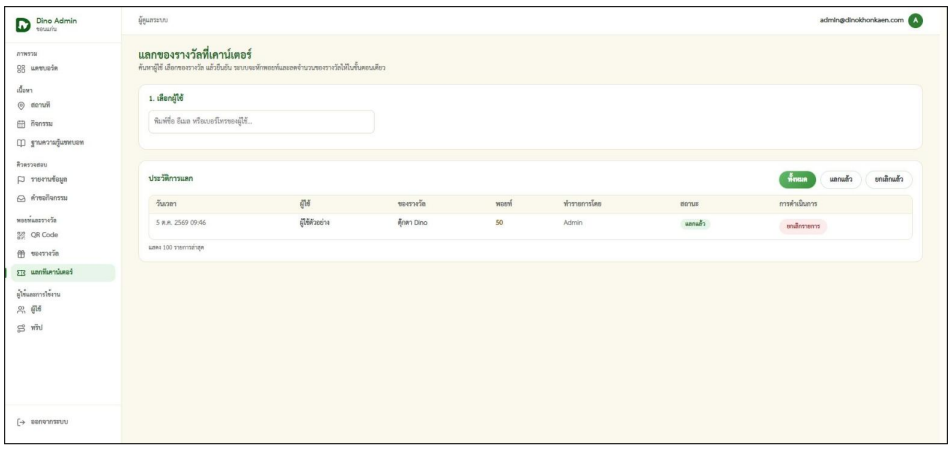
ภาพที่ 35 หน้าจัดการ QR Code



ภาพที่ 36 หน้าจัดการของรางวัล

(8) เมนูแลกที่เคาน์เตอร์

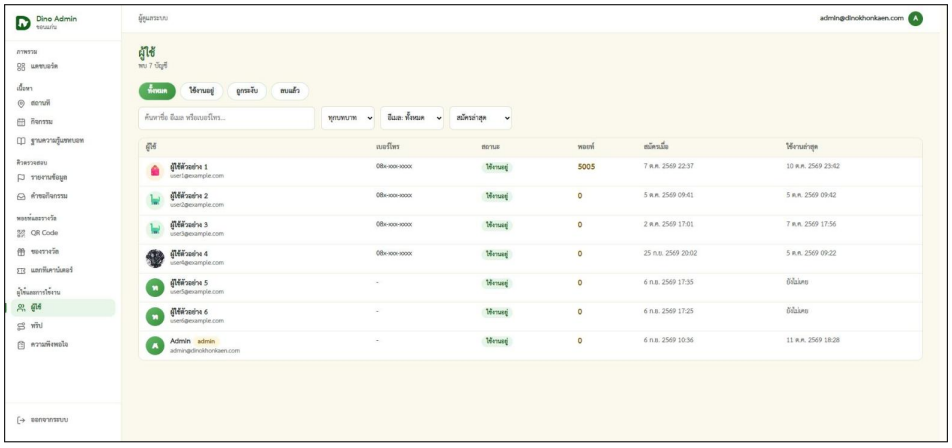
เมื่อสมาชิกมาแลกของรางวัล เจ้าหน้าที่เปิดเมนู “แลกที่เคาน์เตอร์” ดังภาพที่ 37 แล้วดำเนินการ 2 ขั้นตอน คือ เลือกผู้ใช้ และเลือกของรางวัล จากนั้นกดยืนยันเมื่อพร้อมมอบของ ระบบจะหักพอยท์ตามราคาของรางวัล หากบันทึกผิดสามารถยกเลิกรายการจากประวัติได้ ซึ่งระบบจะคืนพอยท์ให้ผู้ใช้และคืนของรางวัล 1 ชิ้นกลับเข้าจำนวนคงเหลือ



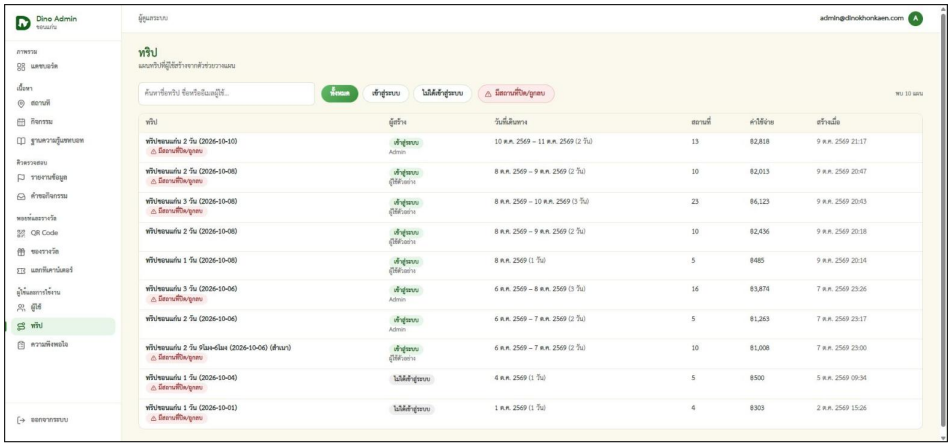
ภาพที่ 37 หน้าแลกของรางวัลที่เคาน์เตอร์

(9) เมนูผู้ใช้ ทริป และความพึงพอใจ

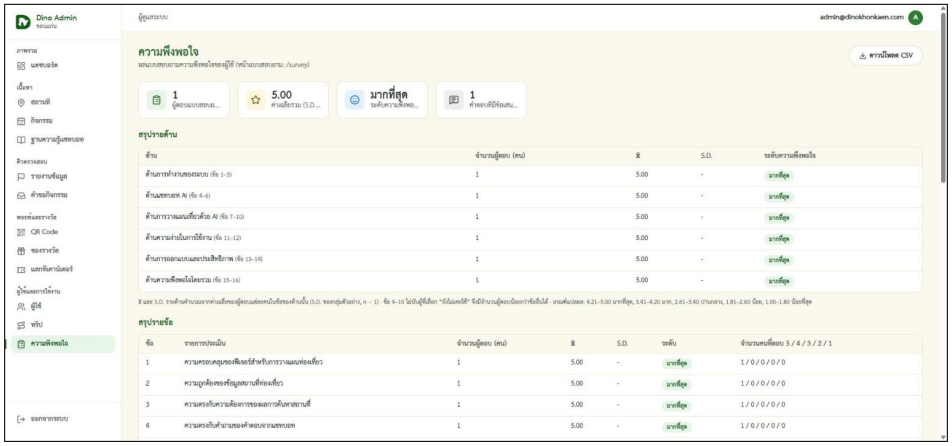
เมนู “ผู้ใช้” ใช้ดูรายละเอียดและจัดการบัญชีผู้ใช้ ดังภาพที่ 38 เมนู “ทริป” ใช้ค้นหาและดูแผนทริปที่ผู้ใช้สร้างไว้ ดังภาพที่ 39 และเมนู “ความพึงพอใจ” แสดงผลแบบสอบถามความพึงพอใจ ได้แก่ คะแนนเฉลี่ย จำนวนผู้ตอบ ข้อเสนอแนะ และคำตอบรายคน ดังภาพที่ 40



ภาพที่ 38 หน้าจัดการผู้ใช้



ภาพที่ 39 หน้าดูทริปของผู้ใช้



ภาพที่ 40 หน้าผลความพึงพอใจ

บทที่ 3

การติดตั้งระบบสำหรับพัฒนา

3.1 ความต้องการของระบบ

3.1.1 ระบบปฏิบัติการ

คู่มือนี้อธิบายการติดตั้งบนระบบปฏิบัติการ Windows 11

3.1.2 ซอฟต์แวร์และบริการที่ต้องเตรียม

(1) Node.js เวอร์ชัน 20 ขึ้นไป (ระบบ CI ใช้เวอร์ชัน 20) สำหรับส่วนติดต่อผู้ใช้งานและบริการข้อมูลหลัก

(2) Python เวอร์ชัน 3.11 สำหรับ AI Service

(3) Git สำหรับดาวน์โหลดซอร์สโค้ดจาก GitHub

(4) Visual Studio Code สำหรับเปิดและแก้ไขโครงงาน

(5) บัญชี Supabase สำหรับฐานข้อมูล PostgreSQL

(6) บัญชี Cloudinary สำหรับจัดเก็บรูปภาพ

(7) บัญชี Gmail ที่เปิดการยืนยันตัวตน 2 ขั้นตอนและสร้าง App Password สำหรับส่งอีเมล

(8) โปรเจกต์ Google Cloud ที่เปิดใช้ Places API (New) และ Maps JavaScript API

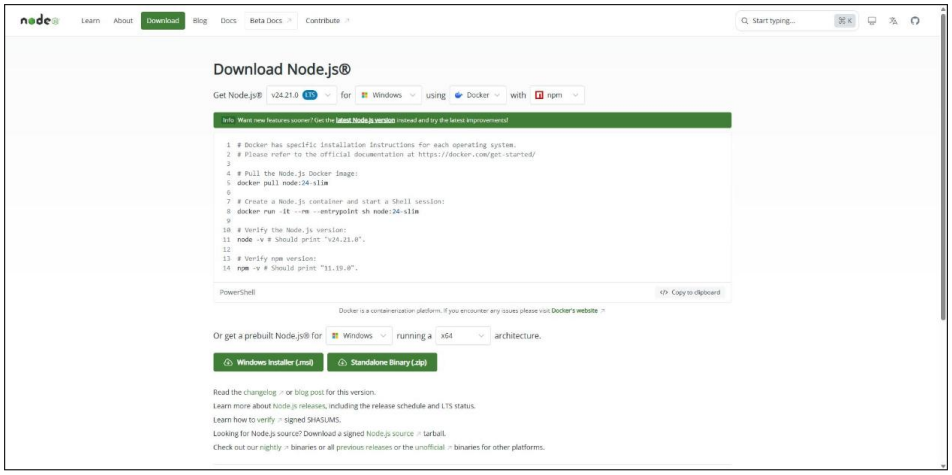
(9) คีย์ KKU AI Gateway สำหรับเรียกใช้โมเดลภาษาขนาดใหญ่

(10) คีย์ Serper API (ไม่บังคับ) สำหรับสคริปต์สร้างคำอธิบายสถานที่

3.2 ขั้นตอนการติดตั้งระบบสำหรับนักพัฒนา

3.2.1 การติดตั้ง Node.js

1. เข้าเว็บไซต์ https://nodejs.org/ แล้วเลือกดาวน์โหลดเวอร์ชัน LTS ดังภาพที่ 41
2. เปิดไฟล์ติดตั้งที่ดาวน์โหลดมา แล้วกด Next จนติดตั้งเสร็จสิ้น
3. เปิด PowerShell แล้วตรวจสอบการติดตั้งด้วยคำสั่งต่อไปนี้ ซึ่งควรแสดงหมายเลขเวอร์ชัน

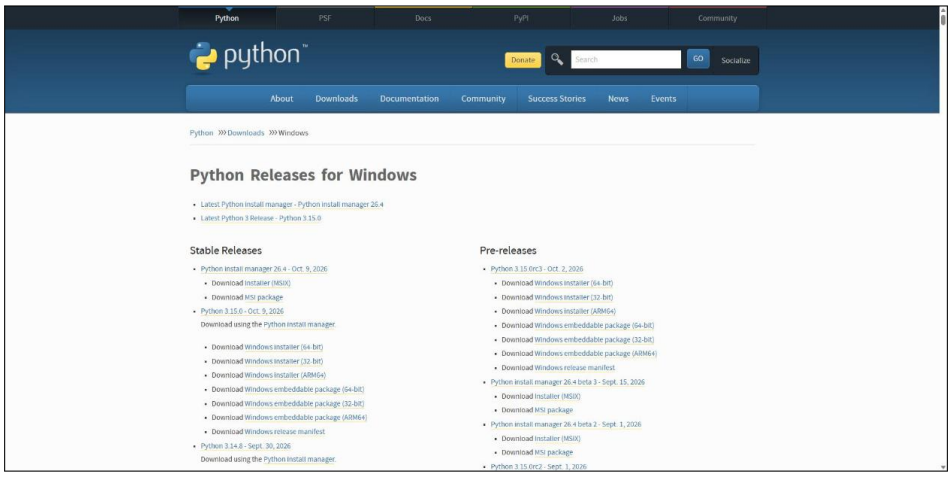


ภาพที่ 41 หน้าดาวน์โหลด Node.js

```
node -v
npm -v
```

3.2.2 การติดตั้ง Python

1. เข้า เว็บไซต์ https:// www.python.org/ downloads/ windows/ แล้วดาวน์โหลดตัวติดตั้ง Windows installer (64-bit) ของรุ่น 3.11 ขึ้นไป ดังภาพที่ 42
2. ขณะติดตั้งให้เลือกตัวเลือก “Add python.exe to PATH” แล้วกด Install Now
3. ตรวจสอบการติดตั้งด้วยคำสั่ง `python --version`



ภาพที่ 42 หน้าดาวน์โหลด Python

3.2.3 การติดตั้ง Visual Studio Code และโครงสร้างโปรเจกต์

ดาวน์โหลด และ ติด ตั้ง Visual Studio Code จาก https://code.visualstudio.com/ แล้วใช้เปิดโฟลเดอร์โปรเจกต์เพื่อเขียนและแก้ไขซอร์สโค้ด

โครงสร้างโปรเจกต์ประกอบด้วย 3 ส่วนหลักที่แยกโฟลเดอร์และทำงานแยกจากกัน ได้แก่ `frontend` `backend` และ `chatbot-service` รวมถึงโฟลเดอร์ `deploy` สำหรับนำระบบขึ้นใช้งานจริง ดังตารางที่ 1

ตารางที่ 1 โครงสร้างโฟลเดอร์ของโครงงาน

โฟลเดอร์	หน้าที่
`frontend`	หน้าเว็บ (React 18 + Vite) ที่ผู้ใช้งานและผู้ดูแลระบบใช้งาน มีโฟลเดอร์ `src` เก็บหน้าเพจ คอมโพเนนต์ และส่วนของผู้ดูแลระบบ
`backend`	REST API (Node.js + Express) โดย `src` เก็บโค้ดของเซิร์ฟเวอร์ `scripts` เก็บสคริปต์นำเข้าข้อมูลและสร้างบัญชีผู้ดูแลระบบ `data` เก็บข้อมูลสถานที่ตั้งต้น `supabase` เก็บ migration ของฐานข้อมูล และ `test` เก็บชุดทดสอบ
`chatbot-service`	AI Service (Python + FastAPI) ได้แก่ แชตบอต ระบบวางแผนทริป และการสร้าง Embedding โดย `src` เก็บโค้ดของบริการ `scripts` เก็บสคริปต์ช่วยงาน และ `tests` เก็บชุดทดสอบ
`deploy`	ไฟล์สำหรับนำระบบขึ้นใช้งานจริง ได้แก่ `docker-compose.yml` `Dockerfile.web` `deploy.sh` ไฟล์ตัวอย่างตั้งค่า `.env` และ `Caddyfile`
`.github/workflows`	ไฟล์กำหนดขั้นตอนตรวจสอบและนำระบบขึ้นอัตโนมัติด้วย GitHub Actions

3.2.4 การดาวน์โหลดซอร์สโค้ดจาก GitHub

เปิด PowerShell ไปยังโฟลเดอร์ที่ต้องการเก็บโครงงาน แล้วคัดลอกซอร์สโค้ดจาก repository ด้วยคำสั่งต่อไปนี้

```
git clone https://github.com/Poeir/dino-trip.git
cd dino-trip
```

3.2.5 การสร้างฐานข้อมูลบน Supabase

1. เข้าเว็บไซต์ https://supabase.com/ ลงชื่อเข้าใช้ แล้วสร้างโปรเจกต์ใหม่ พร้อมบันทึกรหัสผ่านฐานข้อมูลที่กำหนดไว้
2. เปิดหน้า Project Settings > Database > Connection string แล้วคัดลอก Session pooler (พอร์ต 5432) เพื่อนำไปกำหนดเป็นค่า **DATABASE_URL** ไม่ควรใช้ Transaction mode (พอร์ต 6543)

เนื่องจาก backend เป็นเซิร์ฟเวอร์ที่ทำงานต่อเนื่อง

3. เปิดหน้า Project Settings > API แล้วคัดลอก Project URL และ service_role key เพื่อนำไปกำหนดเป็นค่า **SUPABASE_URL** และ **SUPABASE_SERVICE_ROLE_KEY** ซึ่งทั้ง AI Service และสคริปต์นำเข้าข้อมูลของ backend ใช้งาน
4. สร้างโครงสร้างตารางด้วยการนำไฟล์ migration ไปใช้กับฐานข้อมูลใหม่ ตามขั้นตอนในหัวข้อการนำเข้าข้อมูลตั้งต้น

3.2.6 การติดตั้งไลบรารีของแต่ละส่วน

เปิดโฟลเดอร์โครงงานด้วย Visual Studio Code (**code .**) แล้วใช้ Terminal ติดตั้งไลบรารีของแต่ละส่วนตามคำสั่งต่อไปนี้

```
cd backend
npm install

cd ../frontend
npm install

cd ../chatbot-service
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
```

3.2.7 การตั้งค่าไฟล์ .env

ไฟล์ **.env** คือไฟล์ที่ใช้เก็บตัวแปรสภาพแวดล้อม (environment variables) ซึ่งเป็นค่าตั้งต้นที่โปรแกรมอ่านตอนเริ่มทำงาน เช่น ที่อยู่ของฐานข้อมูล รหัสผ่าน คีย์ของบริการภายนอก และที่อยู่ที่แต่ละส่วนของระบบใช้ติดต่อกัน การแยกค่าเหล่านี้ออกจากซอร์สโค้ดทำให้เปลี่ยนค่าตามเครื่องหรือสภาพแวดล้อมที่ใช้งานได้โดยไม่ต้องแก้โปรแกรม และไม่ต้องนำข้อมูลลับไปไว้ในซอร์สโค้ด หากไม่กำหนดค่าที่จำเป็น ส่วนนั้นของระบบจะไม่สามารถเริ่มทำงานหรือเชื่อมต่อบริการที่เกี่ยวข้องได้ ทั้งนี้ ต้องเริ่มการทำงานของระบบใหม่ทุกครั้งหลังแก้ไขไฟล์ **.env**

แต่ละส่วนมีไฟล์ **.env.example** เป็นต้นแบบ ให้คัดลอกเป็นไฟล์ **.env** ในโฟลเดอร์เดียวกัน แล้วกรอกค่าตามตารางที่ 2 ไฟล์ **.env** มีข้อมูลลับ จึงห้ามบันทึกเข้าระบบ Git

```
cd backend
copy .env.example .env
cd ../frontend
copy .env.example .env
cd ../chatbot-service
copy .env.example .env
```

ตารางที่ 2 ตัวแปรสภาพแวดล้อมที่ใช้ในไฟล์ .env

ส่วนของระบบ	ตัวแปร	ความจำเป็น	คำอธิบาย
backend	DATABASE_URL	บังคับ	สายอักขระเชื่อมต่อ (connection string) ของ PostgreSQL ใช้ Session pooler พอร์ต 5432
backend	CLOUDINARY_CLOUD_NAME CLOUDINARY_API_KEY CLOUDINARY_API_SECRET	บังคับ	ค่าจาก Cloudinary Dashboard > Settings > API Keys ใช้อัปโหลดรูปภาพ
backend	GMAIL_USER GMAIL_APP_PASSWORD	บังคับ (ยกเว้นเมื่อใช้ SMTP แทน)	บัญชี Gmail และ App Password สำหรับส่งอีเมลยืนยัน
backend	FRONTEND_ORIGIN	ไม่บังคับ (ต้องกำหนดเมื่อใช้งานจริง)	origin ของ frontend ต้องตรงกันทุกตัวอักษรและไม่มี / ท้าย ค่าเริ่มต้น http://localhost:5173
backend	CHATBOT_SERVICE_URL	ไม่บังคับ (ต้องกำหนดเมื่อใช้งานจริง)	ที่อยู่ของ AI Service ค่าเริ่มต้น http://localhost:8000
backend	PORT	ไม่บังคับ	พอร์ตของ backend ค่าเริ่มต้น 4000
backend	GOOGLE_PLACES_API_KEY	บังคับเฉพาะการนำเข้าและปรับปรุงข้อมูลสถานที่	คีย์ Google Places ต้องเปิดใช้ Places API (New) และผูกบัญชีชำระเงิน
backend	SUPABASE_URL SUPABASE_SERVICE_ROLE_KEY	บังคับเฉพาะการนำเข้าข้อมูลกิจกรรมและฐานความรู้	ใช้กับสคริปต์ seed:events-knowledge ค่าเดียวกับของ AI Service
backend	MAIL_FROM_NAME MAIL_FROM	ไม่บังคับ	ชื่อและที่อยู่ผู้ส่งอีเมล ค่าเริ่มต้นคือ Dino และ GMAIL_USER
backend	SMTP_HOST SMTP_PORT SMTP_USER SMTP_PASSWORD	ไม่บังคับ	ใช้ส่งอีเมลผ่าน SMTP ของผู้ให้บริการอื่นแทน Gmail เมื่อกำหนด SMTP_HOST
frontend	VITE_API_URL	ไม่บังคับ	ที่อยู่ของ backend ค่าเริ่มต้น http://localhost:4000

ตารางที่ 2 ตัวแปรสภาพแวดล้อมที่ใช้ในไฟล์ .env (ต่อ)

ส่วนของระบบ	ตัวแปร	ความจำเป็น	คำอธิบาย
frontend	`VITE_CHATBOT_SERVICE_URL`	ไม่บังคับ	ที่อยู่ของ AI Service ค่าเริ่มต้น `http://localhost:8000`
frontend	`VITE_GOOGLE_MAPS_API_KEY`	ไม่บังคับ (หากไม่กำหนดจะไม่แสดงแผนที่)	คีย์ Google Maps แยกจากคีย์ของ backend ต้องเปิดใช้ Maps JavaScript API และ Places API และควรจำกัดด้วย HTTP referrer
frontend	`VITE_SITE_URL`	ไม่บังคับ	ที่อยู่เว็บไซต์จริงสำหรับแท็กแสดงตัวอย่างลิงก์ ไม่ต้องกำหนดเมื่อพัฒนาในเครื่อง
chatbot-service	`KKU_API_KEY`	บังคับ (เมื่อใช้ KKU AI Gateway)	คีย์ของ KKU AI Gateway
chatbot-service	`SUPABASE_URL` `SUPABASE_SERVICE_ROLE_KEY`	บังคับ	ใช้ค้นข้อมูลสถานที่และค้นหาเชิงเวกเตอร์
chatbot-service	`MODEL_NAME` `TRIP_PLANNER_MODEL_NAME`	ไม่บังคับ (ต้องกำหนดเมื่อใช้งานจริง)	ชื่อโมเดลของแชตบอตและระบบวางแผนทริป มีค่าเริ่มต้นสำหรับการพัฒนา
chatbot-service	`CORS_ORIGINS`	ไม่บังคับ (ต้องกำหนดเมื่อใช้งานจริง)	origin ที่เรียกใช้ AI Service ได้ คั่นด้วย , และไม่มี / ท้าย ค่าเริ่มต้น `http://localhost:5173`
chatbot-service	`SERPER_API_KEY`	ไม่บังคับ	ใช้เฉพาะสคริปต์สร้างคำอธิบายสถานที่
chatbot-service	`LLM_PROVIDER`	ไม่บังคับ	`kku` (ค่าเริ่มต้น) หรือ `vertex` เพื่อใช้ Vertex AI แทน KKU AI Gateway
chatbot-service	`VERTEX_PROJECT` `VERTEX_LOCATION`	บังคับเฉพาะรหัสโปรเจกต์เมื่อใช้ Vertex AI	รหัสโปรเจกต์ (`VERTEX_PROJECT`) และภูมิภาคของ Google Cloud ค่าเริ่มต้นของภูมิภาคคือ `global`

คำว่า “บังคับ” หมายถึงต้องกำหนดค่าก่อนเริ่มการทำงาน ส่วน “ไม่บังคับ” หมายถึงระบบมีค่าเริ่มต้นหรือเป็นฟังก์ชันเสริม ทั้งนี้ ตัวแปรที่ระบุว่าต้องกำหนดเมื่อใช้งานจริง ค่าเริ่มต้นของระบบเหมาะกับการพัฒนาในเครื่องเท่านั้น รายละเอียดเพิ่มเติมอยู่ในไฟล์ `.env.example` ของแต่ละส่วน

3.2.8 การเชื่อมต่อโปรเจกต์ Supabase ด้วย Supabase CLI

ก่อนนำไฟล์ migration ไปใช้ ต้องเชื่อมโฟลเดอร์ **backend** กับโปรเจกต์ Supabase ที่สร้างไว้ โดยใช้ Terminal ที่โฟลเดอร์ **backend** และระบุ project ref ของโปรเจกต์ ซึ่งดูได้จากหน้า Project Settings > General

```
cd backend
npx supabase login
npx supabase link --project-ref <project-ref>
```

3.2.9 การนำเข้าข้อมูลตั้งต้น

ขั้นตอนนี้ดำเนินการเพียงครั้งเดียวตอนติดตั้งฐานข้อมูลใหม่ โดยใช้ Terminal ที่โฟลเดอร์ **backend**

```
cd backend
npx supabase db push --yes
npm run fetch:places
npm run import:places
npm run seed:events-knowledge
```

ความหมายของแต่ละคำสั่งมีดังนี้

1. **npx supabase db push --yes** สร้างตารางและฟังก์ชันตามไฟล์ migration ใช้กับฐานข้อมูลใหม่เท่านั้น เนื่องจากฐานข้อมูลที่ใช้งานจริงอยู่แล้วไม่มีประวัติ migration ทำให้คำสั่งนี้ประมวลผล migration เดิมซ้ำทั้งหมด
2. **npm run fetch:places** เรียก ข้อมูล สถาน ที่ จาก Google Places API และ บันทึก ลง ไฟล์ **backend/ data/ places.json** ซึ่งมีค่าใช้จ่ายตามจำนวนสถานที่
3. **npm run import:places** นำเข้าข้อมูลจากไฟล์ดังกล่าวเข้าฐานข้อมูล
4. **npm run seed:events-knowledge** เพิ่มข้อมูลกิจกรรมและฐานความรู้ตั้งต้น

หลังนำเข้าข้อมูลใหม่ทุกครั้ง ต้องสร้าง Embedding ใหม่ เพื่อให้แชตบอตและระบบวางแผนทริปสามารถค้นหาข้อมูลที่เพิ่มเข้ามาได้

```
cd chatbot-service
venv\Scripts\activate
python scripts/embed_content.py
```

3.2.10 การสร้างบัญชีผู้ดูแลระบบ

บัญชีผู้ดูแลระบบสร้างด้วยสคริปต์ที่โฟลเดอร์ backend โดยระบุอีเมลและรหัสผ่านที่ต้องการ

```
cd backend
node scripts/create-admin.js <email> <password>
```

เมื่อสำเร็จจะแสดงข้อความ “Admin account ready” และสามารถเข้าสู่ระบบที่หน้า /admin/login ได้

3.2.11 การเริ่มการทำงานของระบบ

เปิด Terminal ใน Visual Studio Code แยก 3 หน้าต่าง แล้วเริ่มการทำงานของแต่ละส่วนตามคำสั่งต่อไปนี้

```
# Terminal 1: Backend (http://localhost:4000)
cd backend
npm run dev

# Terminal 2: AI Service (http://localhost:8000)
cd chatbot-service
venv\Scripts\activate
uvicorn src.main:app --reload --port 8000

# Terminal 3: Frontend (http://localhost:5173)
cd frontend
npm run dev
```

เมื่อทุกส่วนทำงานแล้ว ให้เปิดเบราว์เซอร์ไปที่ http://localhost:5173 จะปรากฏหน้าแรกของไดโน และสามารถดูเอกสาร API ของ AI Service ได้ที่ http://localhost:8000/docs

3.2.12 ภาพรวมการติดตั้งระบบเพื่อใช้งานจริง

ระบบที่ใช้งานจริงที่ https://dinokhonkaen.app ทำงานบนเซิร์ฟเวอร์เสมือน (VPS) เครื่องเดียว โดยใช้ Docker Compose ในโฟลเดอร์ deploy จัดการบริการทั้งหมดให้ทำงานร่วมกันภายใต้ชื่อโดเมนเดียว ซึ่งทำให้ cookie ของการเข้าสู่ระบบเป็นของเว็บไซต์เดียวกัน (first-party) และไม่ต้องเปิดพอร์ตของ backend หรือ AI Service ออกสู่ภายนอก บริการที่ Docker Compose จัดการมี 3 บริการ ดังตารางที่ 3

ตารางที่ 3 บริการที่ Docker Compose จัดการ

บริการ	การเข้าถึง	หน้าที่
`caddy`	พอร์ต 80 และ 443 (เปิดสู่ภายนอก)	เว็บเซิร์ฟเวอร์ที่ขอใบรับรอง HTTPS ให้อัตโนมัติ ให้บริการหน้าเว็บที่สร้างจากส่วน `frontend` และส่งต่อคำขอไปยัง backend หรือ AI Service
`backend`	ภายในเครือข่าย Docker เท่านั้น (พอร์ต 4000)	REST API ภายใต้เส้นทาง `/api` ซึ่งเชื่อมต่อฐานข้อมูลและ Cloudinary
`chatbot`	ภายในเครือข่าย Docker เท่านั้น (พอร์ต 8000)	AI Service ซึ่งรับผิดชอบแชตบอตและระบบวางแผนทริป

Caddy ส่งต่อคำขอตามเส้นทางที่กำหนดในไฟล์ `deploy/caddy/Caddyfile` ดังนี้ เส้นทาง `/api/*` ส่งไปยัง backend เส้นทาง `/chat/*`, `/trip/llm` และ `/health` ส่งไปยัง AI Service ส่วนเส้นทางอื่นทั้งหมดให้บริการเป็นหน้าเว็บของ React นอกจากนี้ Caddy ส่งคำขอจากโปรแกรมค้นหา (crawler) ไปยัง backend เพื่อให้ได้หน้าที่มีข้อมูลสำหรับ SEO และตั้งค่าส่วนหัวด้านความปลอดภัยของทุกการตอบกลับ

ในการสร้าง Docker image ของแต่ละบริการ ส่วน `backend` และ `chatbot-service` สร้างจาก Dockerfile ในโฟลเดอร์ของตนเอง ส่วน `caddy` สร้างจากไฟล์ `deploy/Dockerfile.web` ซึ่งแบ่งเป็น 2 ขั้นตอน คือ สร้างไฟล์หน้าเว็บด้วย `npm run build` บน Node.js 20 แล้วคัดลอกผลลัพธ์ไปไว้ใน image ของ Caddy

3.2.13 การเตรียมเซิร์ฟเวอร์และไฟล์ตั้งค่า

เซิร์ฟเวอร์ที่ใช้งานต้องติดตั้ง Docker Engine พร้อม Docker Compose และ Git มีชื่อโดเมนหรือชื่อโฮสต์ที่ชี้มายังเซิร์ฟเวอร์ และเปิดพอร์ต 80 และ 443 ไว้ เนื่องจากการสร้าง image ของ AI Service ใช้หน่วยความจำมาก จึงควรมีหน่วยความจำอย่างน้อย 4 GB แล้วดำเนินการดังนี้

1. ดาวน์โหลดซอร์สโค้ดลงเซิร์ฟเวอร์ด้วยคำสั่ง `git clone` แล้วเปิดโฟลเดอร์ `deploy`
2. สร้างไฟล์ตั้งค่า 3 ไฟล์จากไฟล์ต้นแบบในโฟลเดอร์ `deploy` ตามคำสั่งต่อไปนี้

```
cd deploy
cp .env.example .env
cp backend.env.example backend.env
cp chatbot.env.example chatbot.env
```

จากนั้นกรอกค่าในไฟล์ทั้ง 3 ตามตารางที่ 4 ไฟล์เหล่านี้มีข้อมูลลับ จึงห้ามบันทึกเข้าระบบ Git

ตารางที่ 4 ไฟล์ตั้งค่าสำหรับการใช้งานจริงในโฟลเดอร์ deploy

ไฟล์	ตัวแปรหลัก	คำอธิบาย
.env	CHAT_DOMAIN	ชื่อโฮสต์ของเว็บไซต์ ซึ่ง Caddy ใช้ขอใบรับรอง HTTPS (ต้องกำหนด)
	OLD_DOMAIN	ชื่อโฮสต์เดิมที่ต้องการเปลี่ยนเส้นทางมายังชื่อใหม่ (ไม่บังคับ)
	VITE_GOOGLE_MAPS_API_KEY	คีย์ Google Maps ที่ฝังในหน้าเว็บตอนสร้างภาพ (ไม่บังคับ แต่หากไม่กำหนดจะไม่แสดงแผนที่)
backend.env	DATABASE_URL CLOUDINARY_* GMAIL_* GOOGLE_PLACES_API_KEY	ค่าเดียวกับไฟล์ backend/.env ตามตารางที่ 2 ยกเว้น PORT, FRONTEND_ORIGIN และ CHATBOT_SERVICE_URL ซึ่ง Docker Compose กำหนดให้เอง
chatbot.env	SUPABASE_URL SUPABASE_SERVICE_ROLE_KEY KKU_API_KEY MODEL_NAME TRIP_PLANNER_MODEL_NAME CORS_ORIGINS	ค่าเดียวกับไฟล์ chatbot-service/.env โดยต้องกำหนดชื่อโมเดลทั้งสองเสมอ มิฉะนั้น KKU AI Gateway จะปฏิเสธคำขอ และ CORS_ORIGINS ต้องเป็น origin ของเว็บไซต์จริง เช่น https://dinokhonkaen.app

หาก เปลี่ยน ไป ใช้ Vertex AI แทน KKU AI Gateway ให้ กำหนด LLM_PROVIDER=vertex และ VERTEX_PROJECT ใน chatbot.env และ นำไฟล์กุญแจของ service account วางในโฟลเดอร์ deploy/secrets ซึ่งไม่ถูกบันทึกเข้า Git แล้วเมานต์เข้าบริการ chatbot แบบอ่านอย่างเดียว ตามคำอธิบายในไฟล์ docker-compose.yml

3.2.14 การสร้างและเริ่มระบบด้วย Docker Compose

เมื่อ กำหนด ค่า ครบ แล้ว ให้ สร้าง ภาพ และ เริ่ม บริการ ทั้งหมด ที่ โฟลเดอร์ deploy

```
cd deploy
docker compose up -d --build
```

การสร้างครั้งแรกใช้เวลาค่อนข้างนาน เนื่องจากต้องดาวน์โหลดไลบรารีและโมเดลสำหรับสร้าง Embedding หลังเริ่มบริการแล้ว สามารถตรวจสอบสถานะและบันทึกการทำงานได้ด้วยคำสั่งต่อไปนี้

```
docker compose ps
docker compose logs --tail 50 chatbot
docker compose logs --tail 50 backend
```

จากนั้นเปิดเบราว์เซอร์ไปที่ https://<CHAT_DOMAIN>/health

หากบริการ AI Service พร้อมใช้งานจะตอบกลับสถานะปกติ ทั้งนี้ หลังเริ่มบริการใหม่ AI Service ต้องใช้เวลาโหลดโมเดลก่อน จึงอาจยังไม่ตอบกลับทันที

ระบบจำกัดขนาดบันทึกการทำงานของแต่ละบริการไว้ที่ 3 ไฟล์ ไฟล์ละไม่เกิน 10 MB เพื่อป้องกันพื้นที่จัดเก็บเต็ม และตั้งให้บริการทุกตัวเริ่มทำงานใหม่เองเมื่อหยุดโดยไม่ได้ตั้งใจ (`restart: unless-stopped`)

3.2.15 การปรับปรุงระบบด้วยสคริปต์ deploy.sh

เมื่อมีการแก้ไขโค้ดและต้องการปรับปรุงระบบบนเซิร์ฟเวอร์ ให้ใช้สคริปต์ `deploy/deploy.sh` ซึ่งดำเนินการตามลำดับดังนี้

1. ดึงโค้ดล่าสุดของสาขา `main` จาก GitHub โดยอนุญาตเฉพาะการรวมแบบ fast-forward
2. อ่านเลขเวอร์ชันจากไฟล์ `VERSION` และรหัส commit เพื่อแสดงในหน้าผู้ดูแลระบบ
3. สร้าง image ของบริการ `chatbot`, `backend` และ `caddy` ทีละบริการ เพื่อป้องกันไม่ให้หน่วยความจำของเซิร์ฟเวอร์ไม่เพียงพอ
4. เริ่มบริการด้วย `docker compose up -d` แล้วสั่งให้ Caddy อ่านไฟล์ตั้งค่าใหม่
5. ตรวจสอบที่ `/health` ซ้ำไม่เกิน 3 นาที หากสำเร็จจะแสดงข้อความ “deploy ok” แต่หากไม่สำเร็จจะแสดงบันทึกการทำงานของ `chatbot` ล่าสุดและจบด้วยสถานะผิดพลาด

```
bash deploy/deploy.sh
```

ข้อควรระวัง สคริปต์นี้ไม่ได้นำไฟล์ migration ของฐานข้อมูลไปใช้ หากมีการเปลี่ยนโครงสร้างตาราง ต้องนำไฟล์ migration ใหม่ไปใช้กับฐานข้อมูลก่อนด้วยตนเอง ตามที่อธิบายในหัวข้อการนำเข้าข้อมูลตั้งต้น

3.2.16 การตรวจสอบและนำระบบขึ้นอัตโนมัติด้วย GitHub Actions

โครงงานใช้ GitHub Actions ซึ่งกำหนดไว้ในโฟลเดอร์ `.github/workflows` แบ่งเป็น 2 ส่วน ดังตารางที่ 5

ตารางที่ 5 ขั้นตอนอัตโนมัติของ GitHub Actions

ไฟล์	เงื่อนไขที่ทำงาน	สิ่งที่ดำเนินการ
`ci-frontend.yml`	คำขอรวมโค้ด (pull request) หรือการรวมเข้าสาขา `main` ที่แก้ไขโฟลเดอร์ `frontend`	ติดตั้งไลบรารีด้วย `npm ci` และสร้างไฟล์ด้วย `npm run build` เพื่อตรวจว่าสร้างได้สำเร็จ
`ci-chatbot.yml`	คำขอรวมโค้ดหรือการรวมเข้าสาขา `main` ที่แก้ไขโฟลเดอร์ `chatbot-service`	ติดตั้ง Python 3.11 และไลบรารี แล้วเรียกใช้ชุดทดสอบ `pytest` โดยใช้ค่าจำลองแทนคีย์จริง
`deploy-chatbot.yml`	การรวมเข้าสาขา `main` ที่แก้ไขโฟลเดอร์ `frontend`, `backend`, `chatbot-service` หรือ `deploy` หรือสั่งเริ่มด้วยตนเอง	เชื่อมต่อเซิร์ฟเวอร์ผ่าน SSH เพื่อสั่งให้ `deploy.sh` ปรับปรุงระบบทั้งหมด

ก่อนใช้งานการนำระบบขึ้นอัตโนมัติ ต้องเพิ่มค่าลับ (secrets) ใน GitHub ที่หน้า Settings > Secrets and variables > Actions ของ repository ตามตารางที่ 6

ตารางที่ 6 ค่าลับของ GitHub Actions สำหรับการนำระบบขึ้นอัตโนมัติ

ชื่อค่าลับ	คำอธิบาย
`VPS_HOST`	ชื่อโฮสต์หรือที่อยู่ของเซิร์ฟเวอร์
`VPS_USER`	ชื่อผู้ใช้ที่ใช้เชื่อมต่อ SSH
`VPS_SSH_KEY`	กุญแจส่วนตัว (private key) ของคู่กุญแจที่สร้างขึ้นสำหรับ CI เท่านั้น
`VPS_KNOWN_HOSTS`	ผลลัพธ์ของคำสั่ง `ssh-keyscan -t ed25519` ตามด้วยค่า `VPS_HOST` สำหรับยืนยันตัวตนของเซิร์ฟเวอร์

กุญแจ สาธารณะ (public key) ของ คู่ กุญแจ ดัง กล่าว ต้อง เพิ่ม ใน ไฟล์ `~/.ssh/authorized_keys` บนเซิร์ฟเวอร์ โดยกำหนดคำสั่งบังคับ (forced command) ให้ดำเนินการได้เฉพาะ `deploy.sh` เพื่อให้แม้กุญแจรั่วไหลก็ไม่สามารถเปิดเชลล์เข้าเครื่องได้ รูปแบบของบรรทัดดังกล่าวอยู่ในความคิดเห็นต้นไฟล์ `deploy/deploy.sh`

เมื่อตั้งค่าครบ ทุกครั้งที่มีการรวมโค้ดที่เกี่ยวข้องเข้าสาขา `main` ระบบจะปรับปรุงเว็บไซต์โดยอัตโนมัติ ทั้งนี้ ขั้นตอนตรวจสอบ (CI) และขั้นตอนนำระบบขึ้น (deploy) เป็นคนละส่วนที่ทำงานแยกกัน การนำระบบขึ้นจึงไม่รอผลการตรวจสอบ ผู้พัฒนาควรตรวจผลการทดสอบให้ผ่านก่อนรวมโค้ดเข้าสาขา `main`